

Dart 6: Valores nulos

- Dart 6: Valores nulos
 - Valores nulos
 - Tipos anulables y tipos no anulables
 - Gestionando valores nulos
 - Operadores para valores nulos
 - Operador if-null (??)
 - Operador de asignación para valores nulos (??=)
 - Operador de acceso para nulos / Operador de invocación de métodos para nulos (?.)
 - Null-assertion operator (!)
 - Operador en cascada para nulos (?...)
 - Inicialización de propiedades no nulas
 - Usando inicializadores
 - Usando la forma corta del constructor personalizado
 - Usando una lista de inicialización
 - Usando parámetros con valores por defecto
 - Usando parámetros con nombre requeridos
 - La palabra reservada late
 - Ventajas de usar late
 - Mini ejercicios

Valores nulos

Nulo significa **ausencia de valor** y es un concepto muy útil en programación. Es muy beneficioso tener un valor dedicado `null` que todo el mundo entiende que significa **sin valor**.

Veamos qué ocurre si no se tiene. Imaginemos un programa en el que hay que trabajar con códigos postales. Se podría declarar una variable `codigoPostal` así:

```
int codigoPostal = 23071; // Es el código postal de la Universidad de Jaén
```

¿Qué ocurriría si para un caso concreto, no hay código postal, o en este momento no se tiene? El código, tal y como está, requiere que se asigne un valor. ¿Qué valor se asigna? ¿Tal vez `0` o `-1`?

```
int codigoPostal = -1;
```

El fragmento de código anterior compila y ejecuta correctamente, pero dado que asignar el valor `-1` en el caso de que no se tenga código postal o no se conozca en este momento, es una decisión arbitraria, es necesario comunicar al resto de posibles usuarios del código el significado. Por otro lado, si se tiene un valor `null` dedicado, que todo el mundo puede entender que significa **sin valor**, el código es mucho más sencillo de escribir y entender por todo el mundo.

Pero, si se escribe en Dart:

```
int codigoPostal = null;
```

se produce el siguiente error de compilación:

```
Error: The value 'null' can't be assigned to a variable of type 'int' because 'int' is not nullable.
```

Importante: En versiones de Dart anteriores a la 2.12, lanzada al mercado en marzo de 2021, la declaración anterior funcionaba correctamente.

Ten cuidado si consultas o descargas algún tutorial de Dart anterior a esa fecha porque puedes tener estos errores.

¿Por qué Dart prohíbe asignar el valor `null` a un tipo como por ejemplo `int` o algún otro de los que hemos visto? La razón es que genera muchos errores en tiempo de ejecución mucho más difíciles de localizar y corregir que los errores en tiempo de compilación.

En Dart, `null` no es un valor, sino que **es un objeto**, la única instancia de una clase llamada `Null`. Esa clase no tiene los métodos o getters que sí tienen otras clases, por eso en la versión actual de Dart se prohíbe asignar el valor `null` a variables definidas de otras clases.

Así se evitan errores de ejecución como:

```
NoSuchMethodError
```

Entonces, si poder usar el objeto `null` es bueno porque es una forma universalmente entendida de expresar **sin valor**, pero no se puede asignar ese objeto a variables declaradas de otras Clases ¿Cómo se usan nulos en Dart?

Tipos anulables y tipos no anulables

Las versiones actuales de Dart, distinguen tipos anulables (admiten el valor `null`) y tipos no anulables (no admiten el valor `null`).

Nota: Recuerda que en Dart, todo son clases, hasta `int` o `double` que en otros lenguajes son tipos simples. Por eso, cuando hablamos de tipos anulables o no anulables, en realidad se habla de clases que pueden contener al objeto `null` o no.

En Dart, todos los tipos son **no anulables por defecto**. Eso significa que el lenguaje **garantiza** que no se le va a asignar el objeto `null`.

```
int noAnulable = 1;
noAnulable = null; // ERROR DE COMPILACIÓN. No se puede asignar null al tipo int
```

Los tipos anulables son aquellos que pueden contener al objeto `null` además de los valores propios del tipo. Esos tipos se distinguen fácilmente, ya que al final de su nombre de clase tiene un signo de interrogación `?`.

Así por ejemplo, `int` es la clase no anulable, no puede contener al objeto `null` e `int?` es la clase anulable, que sí puede contener al objeto `null`.

```
int? anulable = 1;
anulable = null;
print(anulable);
```

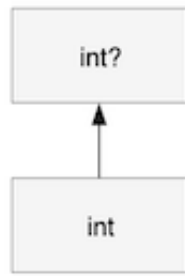
El fragmento de código anterior imprime en la consola de depuración:

```
null
```

Como puede verse, a un tipo que acepta valores nulos como `int?` se le pueden asignar tanto valores propios del tipo como el valor `null`.

Importante: El signo de interrogación `?` al final de un tipo que acepta valores nulos, no es una especie de operador que transforma el tipo original. `int` es un tipo distinto a `int?`.

La versión no nula de un tipo es una **subclase** de la versión con nulos de ese tipo. Así por ejemplo `int` es una subclase de `int?`.



Cualquier clase en Dart, incluyendo las clases que declaremos nosotros, **tiene su correspondiente clase con valores nulos**. Por ejemplo: `int` tiene a `int?`, `String` tiene a `String?` y `Usuario` tiene a `Usuario?`.

Si una variable de un tipo con valores nulos no se inicializa, **su valor por defecto es `null`**

Por ejemplo, el siguiente fragmento de código:

```
int? edad;  
String? nombre;  
  
print(edad);  
print(nombre);
```

imprime en la consola de depuración:

```
null  
null
```

Gestionando valores nulos

Como sabemos, el gran problema de los valores `null` es olvidarse de gestionarlos correctamente. Sin embargo, Dart nos echa una mano. Por ejemplo, el siguiente fragmento de código:

```
String? nombre;  
print(nombre.length);
```

produce el error de compilación:

```
Error: Property 'length' cannot be accessed on 'String?' because it is potentially null.
```

Dart no nos deja ejecutar ese fragmento de código. El **analizador de Dart** comprueba el código y detecta que la variable `nombre` tiene el valor `null` por lo que produce un error **en tiempo de compilación**.

Recuerda: Los errores de compilación son tus aliados, ya que son muy fáciles de detectar y arreglar. Siempre es preferible obtener un error en tiempo de compilación que obtenerlo en tiempo de ejecución.

Sin embargo, si modificamos el fragmento de código anterior y lo cambiamos por el siguiente:

```
String? nombre;  
nombre = 'Francisco';  
print(nombre.length);
```

ahora se ejecuta correctamente e imprime en la consola de depuración el valor `9`.

Eso es así, porque el analizador de Dart es capaz de realizar **promoción de tipos**. Aunque la variable `nombre` es de tipo `String?` que es un tipo anulable y podría contener el valor `null`, Dart sabe que en este caso, no lo contiene, por tanto deja ejecutar el código sin problemas.

Dart nos libera de procesar `nombre` para extraer su valor como un `String`. Eso lo hace Dart automáticamente. A este mecanismo se le llama **promoción de tipos**.

La promoción de tipos funciona incluso en situaciones más complejas como:

```
bool esPositivo(int? unEntero) {  
    if (unEntero == null) {  
        return false;  
    }  
    return !unEntero.isNegative;  
}
```

En este caso, Dart nos deja ejecutar ese código sin problemas, porque sabe que se ha comprobado primero que el valor no sea nulo y se ha tratado adecuadamente. Dart implementa **análisis de flujo**, un mecanismo que comprueba cada posible ruta que el código haya podido seguir y si en ninguna de ellas se tiene el valor `null` realiza la promoción de tipos. Como vemos, Dart es un lenguaje muy potente que facilita la vida al programador.

Por ejemplo, si se ejecuta:

```
print(esPositivo(null));  
print(esPositivo(6));  
print(esPositivo(-2));
```

se obtiene:

```
false  
true  
false
```

Es un fragmento de código que usa nulos de forma completamente segura (**sound null safety**).

Operadores para valores nulos

Dart nos proporciona una serie de operadores para simplificar la gestión de valores nulos, que facilitan mucho la tarea del programador.

Operador if-null (??)

El operador if-null `??` significa: *Si el valor de la izquierda no es `null`, usar directamente, y si no, usar el valor de la derecha.*

Por ejemplo:

```
String? mensaje;  
final texto = mensaje ?? 'Error';  
print(texto);
```

En este ejemplo, `mensaje` tiene valor `null` ya que no se ha inicializado, por eso, como el valor a la izquierda del operador `??` es nulo, se usa el valor de la derecha, la cadena `'Error'`. Por tanto, se imprime en la consola de depuración:

```
Error
```

Hay que tener en cuenta dos cosas:

- Ya que `mensaje` es `null`, a la variable `texto` no se le asigna su valor, sino la cadena `'Error'`.
- Al usar el operador `??` nos estamos asegurando de que `texto` nunca va a contener el valor `null`. Por eso, la inferencia de tipos asume que la variable `texto` es de tipo `String` y no de tipo `String?`.

El uso del operador `??` es equivalente a:

```
String? mensaje;  
String texto;  
if (mensaje == null) {  
    texto = 'Error';  
} else {  
    texto = mensaje;  
}  
print(texto);
```

que es un fragmento de código más largo de escribir y más confuso.

Operador de asignación para valores nulos (??=)

En el caso anterior, se tienen dos variables `mensaje` y `texto`. Sin embargo, hay veces en las que solo se tiene una variable que se quiere actualizar si su valor es `null`.

Por ejemplo, si se tiene la siguiente declaración:

```
double? tamanoTexto;
```

y a la variable `tamanoTexto` no se le ha dado valor, a la hora de usarla se debe comprobar si su valor es nulo, y si no, asignarle un valor por defecto.

Eso se puede conseguir así:

```
tamanoTexto = tamanoTexto ?? 20.0;
```

sin embargo es más cómodo usando el operador de asignación para nulos:

```
tamanoTexto ??= 20.0;
```

Si `tamanoTexto` es `null` se le asigna el valor `20.0` y en caso contrario retiene su valor.

Importante: Tanto `??` como `??=` son operadores útiles para inicializar variables cuando se quiere **garantizar** un valor no nulo.

Operador de acceso para nulos / Operador de invocación de métodos para nulos (?.)

Cuando se quiere acceder a una propiedad de una clase en una instancia, pero esa instancia es `null` se produce un error, ya que la clase `Null` no tiene esa propiedad y por tanto no puede accederse. Eso causa un error fatal que interrumpe la ejecución de la aplicación:

```
NoSuchMethodError
```

Una forma de minimizar el daño de acceder a una propiedad de una clase a través de una instancia que podría ser `null` es usar el operador `?.`

Ese operador devuelve `null` si lo que hay a su izquierda es `null`, si no, devuelve la propiedad que hay a su derecha.

Por ejemplo, el siguiente fragmento de código:

```
int? edad;  
print(edad?.isNegative);
```

se compila sin errores y no produce ningún error en tiempo de ejecución. En este ejemplo, el resultado es que se imprime en la consola de depuración:

```
null
```

El operador de acceso para nulos no solo sirve para propiedades, también sirve para invocar métodos. Por ejemplo, el siguiente fragmento de código:

```
int? edad;  
print(edad?.toDouble());
```

también imprime en la consola:

```
null
```

Ya que como `edad` es `null` no se intenta acceder al método `toDouble()` sino que se devuelve `null`.

Null-assertion operator (!)

En ocasiones Dart no puede determinar si un valor es nulo o no en tiempo de compilación. En estas circunstancias, por seguridad, si se intenta acceder variable de una clase anulable, lanza un error de compilación.

Si estamos **completamente seguros** de que el valor que hay en la variable no es `null` se puede anular este comportamiento predeterminado mediante el operador `!`.

Nota: El null-assertion operator es un **operador sufijo**, siempre va detrás del valor al que se aplica, por lo que no hay confusión posible con el operador lógico `!` que es prefijo y va delante.

Veamos con un ejemplo sencillo la necesidad de uso de este operador. Supongamos la siguiente declaración:

```
String? cadenaAnulable;  
// - Aquí otras instrucciones del programa  
String cadenaNoAnulable = cadenaAnulable;
```

Si el análisis de flujo de Dart no puede determinar si en el momento de ejecutar la última instrucción en `cadenaAnulable` hay un `String` normal o `null`, se produce un error de compilación que impide hacer la asignación.

Sin embargo, puede ser que estemos completamente seguros de que en `cadenaAnulable` sí que hay un `String` y no el valor `null`. En ese caso, se puede forzar a Dart a aceptar la asignación.

Para ello se puede escribir:

```
String? cadenaAnulable;  
// - Aquí otras instrucciones del programa  
String cadenaNoAnulable = cadenaAnulable!;
```

Veamos ahora un ejemplo un poco más elaborado. Tenemos el siguiente fragmento de código:

```
bool? esUsuarioSistema(int id) {  
    if ((id >= 0) && (id < 1000)) {  
        return true;  
    } else if (id >= 1000) {  
        return false;  
    }  
    return null;  
}
```


es una función que, dado un `id` de tipo `int` indica si es usuario de sistema ($0 \leq id < 1000$), usuario normal ($id \geq 1000$) o usuario nulo en cualquier otro caso. Dado que unas veces devuelve un valor booleano `true` o `false` y otras veces devuelve `null`, Dart no es capaz de determinar si la siguiente instrucción se va a ejecutar sin errores o no (para ello tendría que entender la lógica de la función):

```
bool esSistema = esUsuarioSistema(10);
```

por esa razón produce el error de compilación:

```
Error: A value of type 'bool?' can't be assigned to a variable of type 'bool' because 'bool?' is nullable and 'bool' isn't.
```

En el ejemplo concreto, sabemos que el valor `10` no corresponde con un valor `null` por tanto podríamos escribir la instrucción así:

```
bool esSistema = esUsuarioSistema(10)!;
```

con lo que se elimina el error.

Precaución: Usar el operador `!` (null-assertion operator) hará que se termine la ejecución de la aplicación con un error en tiempo de ejecución si el valor realmente resulta ser `null`, así que no hay que usar el operador `!` a menos que podamos estar completamente seguros de que la variable no es `null`.

Un enfoque más seguro, que no produce el bloqueo de la aplicación en ningún caso es:

```
bool esSistema = esUsuarioSistema(-1) ?? false;
```

El valor `-1`, no es ningún caso tratado por la función `esUsuarioSistema`, pero aunque el valor devuelto es `null` el programa lo gestiona y permite seguir con la ejecución. Se ha elegido el valor `false` porque un usuario nulo tampoco es usuario del sistema así que es la opción más lógica.

Operador en cascada para nulos (`?..`)

Cuando se estudiaron las clases, se vio que existía un operador `..` (el operador de asignación en cascada) que simplificaba la asignación de valores a las propiedades de una clase.

Por ejemplo, si se tiene la clase `Usuario` definida de esta forma:

```
class Usuario {  
  int? id;  
  String? nombre;  
}
```

sus propiedades se pueden inicializar así:

```
final usuario = Usuario()  
  ..id = 1234  
  ..nombre = 'Francisco';
```

Cuando se trata de un objeto que puede ser nulo, entonces hay que usar el operador en cascada especial para valores nulos `?..`

```
Usuario? usuario;  
usuario  
  ?..id = 1234  
  ..nombre = 'Francisco';
```

Nota: El signo `?` solo hay que ponerlo en la primera línea, ya que si `usuario` es `null` la inicialización se termina, sin llamar a los otros elementos en la cascada.

Inicialización de propiedades no nulas

Cuando se instancia una clase, Dart obliga a que todas las propiedades no nulas de esa clase estén inicializadas antes de poder usar la instancia.

Por ejemplo, si se tiene la clase:

```
class Usuario {  
  String nombre;  
}
```

dado que la propiedad `nombre` es de clase `String` y no `String?`, debe ser inicializada de algún modo antes de poder usar instancias de `Usuario`. Veamos las distintas formas que Dart proporciona para hacerlo.

Nota: Todas ellas ya se han visto en la sesión 1 de prácticas. Aquí hay un pequeño resumen.

Usando inicializadores

Una forma de inicializar una propiedad de una clase es usar un **valor inicializador**.

```
class Usuario {  
  String nombre = 'anónimo';  
}
```

A continuación de la declaración de la variable se escribe el signo `=` y el valor con el que se quiere inicializar. De esta forma Dart sabe que la variable no anulable `nombre` siempre va a tener un valor.

Usando la forma corta del constructor personalizado

Como sabemos en los constructores personalizados se puede usar una forma corta que usa la palabra reservada `this`:

```
class Usuario {  
    String nombre;  
    Usuario(this.nombre);  
}
```

En este caso, la única forma de construir instancias de la clase `Usuario` es mediante el constructor generativo personalizado (recuerda que si se escribe, entonces Dart no añade el constructor por defecto). Dado que el parámetro del constructor personalizado no es opcional, hay que escribirlo siempre al instanciar la clase, lo que garantiza que la propiedad `nombre` siempre va a tener valor.

Usando una lista de inicialización

Si la propiedad es privada, no se puede usar la versión corta del constructor personalizado, pero se puede usar una lista de inicialización:

```
class Usuario {  
    String _nombre;  
    Usuario(String nombre):  
        _nombre = nombre;  
}
```

Como en el caso anterior, el constructor generativo personalizado es la única forma de construir instancias de `Usuario` y dado que su parámetro no es opcional, siempre debe escribirse en la construcción.

Usando parámetros con valores por defecto

En otros casos, los parámetros del constructor generativo personalizado se marcan como opcionales. Pero en estos casos, Dart obliga a escribir un valor por defecto para esos parámetros, con lo que si no se escribe el argumento correspondiente en la llamada al constructor, Dart toma el valor por defecto y la propiedad siempre tiene valor.

En el caso de parámetros opcionales sin nombre se escribe así:

```
class Usuario {  
    String nombre;  
    Usuario([this.nombre = 'anónimo']);  
}
```

Y en el caso de parámetros opcionales con nombre se escribe así:

```
class Usuario {
  String nombre;
  Usuario({this.nombre = 'anónimo'});
}
```

Usando parámetros con nombre requeridos

Si se quiere usar parámetros con nombre, pero que no sean opcionales, se usa la palabra reservada `required`:

```
class Usuario {
  String nombre;
  Usuario({required this.nombre});
}
```

En este caso, siempre hay que escribir el argumento correspondiente a `nombre` en la llamada al constructor, por lo que siempre tiene un valor.

La palabra reservada `late`

En ocasiones, las clases tienen propiedades no anulables, pero que no pueden inicializarse de alguna de las formas que se han visto en el apartado anterior.

Por ejemplo, la siguiente clase:

```
class UnaClase {
  String unValor = _realizarCalculosComplejos();
  String _realizarCalculosComplejos() {
    print('Realizando cálculos complejos');
    return 'Valor resultado de los cálculos complejos';
  }
}
```

inicializa su propiedad `unValor` con el resultado de un método llamado `_realizarCalculosComplejos`, sin embargo, no funciona. Se produce el siguiente error de compilación:

```
Error: Can't access 'this' in a field initializer to read '_realizarCalculosComplejos'.
```

Eso se debe a que **Dart no permite el acceso a métodos de instancia durante la inicialización** de una instancia (porque podrían acceder a otras propiedades aún no inicializadas)

Para eliminar el error de compilación se puede usar la palabra reservada `late` que se escribe justo delante de la declaración de la propiedad:

```
late String unValor = _realizarCalculosComplejos();
```

La palabra reservada `late` significa que Dart no inicializa la propiedad de inmediato. Solo la inicializa cuando se acceda a ella por primera vez. Esto también se conoce como **inicialización diferida**.

Ventajas de usar late

El uso de la palabra reservada `late`, además de eliminar el error de compilación, tiene otra ventaja importante.

En el ejemplo, al inicializar la propiedad `unValor` es necesario realizar cálculos complejos que podrían requerir bastante tiempo. Marcar esa propiedad como `late` es bueno ya que hace que la inicialización **se posponga** hasta que realmente se vaya a usar la propiedad.

Esta técnica, permite ahorrar tiempo de carga de la aplicación móvil, tiempo de respuesta al pasar de pantalla, etc.

Si se escribe en la función `main`:

```
var unObjeto = UnaClase();
```

vemos que en la consola de depuración no se imprime el mensaje:

```
Realizando cálculos complejos
```

lo que prueba que la propiedad `unValor` no se ha inicializado al construir la instancia de `UnaClase`.

Sin embargo, si a continuación se escribe:

```
print(unObjeto.unValor);
```

ahora se obtiene en la consola los mensajes:

```
Realizando cálculos complejos  
Valor resultado de los cálculos complejos
```

Es decir, la primera vez que se intenta acceder a la propiedad `unValor`, Dart inicializa la propiedad y luego la usa. **Se ha diferido la inicialización de la propiedad** hasta el momento justo en que se usa por primera vez. Si en alguna ejecución concreta de la aplicación, no se necesita acceder a `unValor` entonces no se inicializa nunca, con el consiguiente ahorro de cálculo.

Si a continuación se vuelve a ejecutar:

```
print(unObjeto.unValor);
```

vemos que en la consola de depuración solo se obtiene el mensaje:

```
Valor resultado de los cálculos complejos
```

Como es lógico, la propiedad solo se inicializa una vez, la primera vez que se usa, el resto de usos no causan que se inicialice de nuevo.

Mini ejercicios

1. Intenta averiguar, sin programarlo, el resultado de la siguiente instrucción:

```
print(null.runtimeType);
```

Nota: Se habla de `runtimeType` en la sección: **Comprobación de tipos en tiempo de ejecución**.

Ahora prográmalo y compara la salida en la consola con tu respuesta.

2. Se tiene el siguiente fragmento de código:

```
const unaConstante = 'Dart rocks!';
```

Averigua, sin programarlo, cuál es el tipo que Dart infiere para la constante `unaConstante`.
Explica ¿por qué?

3. ➡ Transforma la clase `Usuario` (la versión que aparece en la sección **Miembros de clase**) para que pueda ser anulable (se le pueda asignar el valor `null`).

Con ella define una variable llamada `usuarioNulo` y asígnale el valor `null`.

Explica con detalle qué cambios has tenido que hacer a la clase original.

4. Escribe una función que, de forma aleatoria, devuelva bien el valor 1000, bien el valor `null`. Asigna el resultado de la función a una variable llamada `resultado` asegurando que nunca tiene valor `null`. Usa como valor por defecto `0`.

5. Si la declaración e inicialización de una propiedad que puede contener valores nulos se precede de la palabra reservada `late`, la propiedad no se inicializa hasta que se necesita por primera vez. A esto se le llama inicialización diferida.

¿Qué otro tipo de propiedades, estudiadas en la sesión anterior de prácticas, muestran este comportamiento?

6. ➡ En algunos países del mundo, a la hora de escribir el nombre, se escribe primero el nombre de pila y a continuación los apellidos, mientras que en otros países es justo al contrario. También se da el caso de que no en todo el mundo se usan apellidos.

Escribe una clase llamada `Nombre` con dos propiedades `nombre` y `apellidos`. Esa clase debe gestionar las situaciones descritas al comienzo de este mini ejercicio.

Sobreescribe el método `toString()` para que genere una cadena de caracteres con el nombre de forma correcta.

Dart 7: Colecciones

- DSDM – Práctica 2 - Dart 7: Colecciones
 - Colecciones
 - Listas
 - Crear una lista
 - Imprimir listas
 - Acceder a los elementos de la lista
 - Asignar nuevos valores a los elementos de una lista
 - Añadir elementos al final de una lista
 - Insertar elementos en una lista
 - Eliminar elementos de una lista
 - Ordenar listas
 - Invertir una lista
 - Listas mutables e inmutables
 - Propiedades interesantes de las listas
 - Sublistas
 - Iterar sobre los elementos de una lista
 - Otros operadores para listas
 - Operación de propagación
 - if para colecciones (collection if)
 - for para colecciones (collection for)
 - Listas anulables
 - La propia lista es anulable
 - Los elementos de la lista son anulables
 - Tanto la lista como sus elementos son anulables
 - Uso de operadores para valores nulos en listas
 - Operadores para valores nulos específicos para listas
 - Copiar listas
 - Conjuntos
 - Crear un conjunto
 - Imprimir conjuntos
 - Averiguar si un elemento está en el conjunto
 - Añadir nuevos elementos a un conjunto
 - Añadir varios elementos a un conjunto
 - Eliminar elementos de un conjunto
 - Iterar sobre los elementos de un conjunto
 - Copiar conjuntos
 - Otros operadores para conjuntos
 - Operador de propagación
 - if para colecciones (collection if)

- for para colecciones (collection for)
 - Operaciones booleanas sobre conjuntos
 - Intersección
 - Unión
 - Diferencia
 - Conjuntos anulables
- Mapas
 - Crear un mapa
 - Imprimir un mapa
 - Llaves únicas
 - Acceder a los pares del mapa
 - Añadir pares a un mapa
 - Actualizar un par en un mapa
 - Eliminar pares de un mapa
 - Propiedades interesantes de los mapas
 - Comprobar la existencia de una llave o un valor
 - Iterar sobre los pares de un mapa
 - Copiar mapas
- ¿Cuándo usar cada colección
- Higher order methods
 - Iterar sobre una colección
 - Iterar sobre un mapa
 - Transformar los elementos una colección
 - Filtrar una colección
 - Resumir una colección
 - reduce
 - fold
 - Ordenar listas de forma personalizada
- Mini ejercicios

Colecciones

Prácticamente todos los programas que se escriban en Dart/Flutter van a necesitar manejar colecciones de objetos, que se pueden organizar de múltiples formas. Dart proporciona varias estructuras de datos para resolver esta necesidad. Las tres más importantes son las listas, los conjuntos y los mapas. También se usa mucho, aunque de forma indirecta, otra colección llamada iterable.

Consejo: No se pueden construir las interfaces de las aplicaciones móviles escritas en Flutter, sin usar listas y mapas, así que te recomiendo que estudies bien las secciones siguientes.

Listas

Las listas en Dart son parecidas a los arrays en otros lenguajes de programación. Suelen usarse cuando hay que gestionar una colección grande de objetos **del mismo tipo** y cuando el **orden** de los objetos sea importante.

Como suele ser habitual los índices de las listas comienzan en `0`

El orden de los elementos en la lista es importante. Si se itera sobre una lista varias veces, siempre se van a obtener los mismos elementos en el mismo orden.

Crear una lista

Si se tienen todos los elementos de la lista en tiempo de compilación, se puede construir así:

```
var fabricantes = ['Apple', 'HP', 'Lenovo', 'Toshiba'];
```

Se escriben los elementos iniciales de la lista entre corchetes `[ ]` y separados por comas `,`. A esto se le llama un **literal de lista**.

Dart infiere el tipo de la variable `fabricantes` como `List<String>` o lista de `String` ya que los elementos del literal de lista son de tipo `String`

Una lista vacía se escribe solo con los corchetes sin elementos. Por ejemplo, la siguiente instrucción:

```
List<String> fabricantes = [];
```

asigna a la variable `fabricantes` una nueva lista de `String` vacía.

Si se quiere declarar una variable y asignarle una **lista vacía** se podría escribir:

```
var componentes = [];
```

en este caso, al no haber elementos iniciales a partir de los que inferir el tipo y haber usado `var`, Dart no tiene suficiente información para inferir el tipo de los elementos de la lista, por lo que le asigna tipo `dynamic`. Este tipo no es muy seguro, por lo que hacer esto trae más problemas de los que resuelve y **no se debería usar**.

Importante: Cuando se quiera inicializar una lista vacía, es mejor hacerlo de esta forma:

```
List<String> componentes = [];
```

o bien de esta otra forma más corta:

```
var componentes = <String>[];
```

Cosas importantes respecto a las listas:

- Las listas son instancias de la clase `List`
- Son clases genéricas, cuando se instancian hay que usar `< >` para escoger el tipo concreto de los elementos de la lista.

Los ángulos `< >` en Dart denotan **tipos genéricos** (se estudian en secciones posteriores).

Una lista genérica significa que se puede tener una lista de cualquier cosa; solo hay que poner el tipo que se necesite dentro de los ángulos `< >`. En este ejemplo, se ha definido una lista de cadenas, pero se podría reemplazar `String` por cualquier otro tipo. Por ejemplo, `List<int>` declara una lista de enteros o `List<bool>` una de booleanos.

Imprimir listas

Cualquier colección puede imprimirse directamente. Por ejemplo, si se ejecuta el siguiente fragmento de código:

```
var fabricantes = ['Apple', 'HP', 'Lenovo', 'Toshiba'];  
print(fabricantes);
```

en la consola de depuración se muestra:

```
[Apple, HP, Lenovo, Toshiba]
```

Nota: Cuando se muestra una lista en la consola, sus elementos se escriben entre `[ ]` para indicar que se trata de una lista.

Acceder a los elementos de la lista

Para acceder a un elemento de una lista, se usa su índice escrito entre corchetes `[ ]`, a esto se le llama **notación de subíndice**. El índice entre corchetes se escribe justo después del nombre de la lista. Por ejemplo, las instrucciones:

```
final tercerElemento = fabricantes[2];  
print(tercerElemento);
```

imprime en la consola:

```
Lenovo
```

Importante: No hay que olvidar que los índices de las listas en Dart comienzan en `0`, por lo que el tercer elemento tiene índice `2`

En Dart también es posible averiguar el índice de un elemento de una lista dado su valor. Para ello se usa el método `indexOf` de la clase `List`. Por ejemplo:

```
final indice = fabricantes.indexOf('HP');  
final valor = fabricantes[indice];  
print('En el índice: $indice se encuentra el valor: $valor');
```

imprime en la consola:

```
En el índice: 1 se encuentra el valor: HP
```

Importante: Si el elemento que se pasa al método `indexOf` de la clase `List` no está en la lista, el valor devuelto es `-1`

Asignar nuevos valores a los elementos de una lista

Se puede cambiar el valor de un elemento de lista de la misma manera que se accede a su valor, es decir, utilizando la notación de subíndice. Por ejemplo el siguiente fragmento de código:

```
fabricantes[1] = 'Asus';  
print(fabricantes);
```

hace que se imprima en la consola:

```
[Apple, Asus, Lenovo, Toshiba]
```

Importante: Como es de esperar, si se escribe un índice menor que `0` o mayor o igual que el número de elementos de la lista se produce un error en tiempo de ejecución.

Por ejemplo, si se ejecuta:

```
print(fabricantes[4]);
```

sobre la lista `fabricantes` declarada en el ejemplo anterior, se obtiene el error:

```
RangeError (index): Invalid value: Not in inclusive range 0..3: 4
```

Añadir elementos al final de una lista

Las listas en Dart son de **tamaño variable** y se pueden ampliar. Para ello se usa el método `add`. El siguiente fragmento de código:

```
fabricantes.add('Acer');  
print(fabricantes);
```

imprime en la consola:

```
[Apple, Asus, Lenovo, Toshiba, Acer]
```

Importante: Como es de esperar, el tipo del elemento que se añade debe coincidir con el tipo de los elementos de la lista. En el ejemplo anterior, si se escribe:

```
fabricantes.add(1);
```

produce el siguiente error en tiempo de compilación:

```
Error: The argument type 'int' can't be assigned to the parameter type 'String'.
```

Insertar elementos en una lista

También es posible insertar elementos en cualquier posición de la lista, no solo al final. Para ello se usa el método `insert` de la clase `List`. Por ejemplo, el siguiente fragmento de código:

```
fabricantes.insert(1, 'Dell');  
print(fabricantes);
```

imprime en la consola de depuración:

```
[Apple, Dell, Asus, Lenovo, Toshiba, Acer]
```

El nuevo elemento se coloca en la posición que se pasa como argumento y el resto de los elementos originales se desplazan de posición para dejar sitio al nuevo elemento insertado.

Eliminar elementos de una lista

Las listas en Dart son de tamaño variable, por tanto, también se pueden eliminar elementos. Para ello se usan los métodos `remove` y `removeAt` de la clase `List`.

En el siguiente fragmento de código:

```
fabricantes.remove('Lenovo');  
print(fabricantes);
```

se usa el método `remove` para eliminar el elemento `Lenovo` de la lista. En la consola se imprime:

```
[Apple, Dell, Asus, Toshiba, Acer]
```

Nota: Si el elemento que se pasa como argumento no está en la lista, entonces la orden `remove` no hace nada y la lista queda intacta.

Si se conoce el índice del elemento que se quiere eliminar, se puede usar `removeAt`. El siguiente fragmento de código:

```
fabricantes.removeAt(3);  
print(fabricantes);
```

imprime en la consola:

```
[Apple, Dell, Asus, Acer]
```

Nota: Si se pasa como argumento un entero menor que `0` o mayor o igual que el número de elementos de la lista, se produce un error de índice fuera de rango.

Ordenar listas

Para ordenar una lista en Dart se usa el método `sort` de la clase `List`. Su uso es sencillo. Por ejemplo, si se tiene la siguiente lista desordenada de enteros:

```
final listaEnteros = [32, 73, 2, 343, 7, 10, 1];
```

se puede ordenar ejecutando:

```
listaEnteros.sort();  
print(listaEnteros);
```

En la consola de depuración se imprime:

```
[1, 2, 7, 10, 32, 73, 343]
```

Importante: El método `sort` de las listas no genera una nueva lista ordenada, sino que **ordena la propia lista** sobre la que se aplica.

El método `sort` de la clase `List` funciona con todos los objetos de las clases que implementen la interfaz `Comparable` y tengan redefinido el método `compare` (Las interfaces en Dart se estudian más adelante).

Por esa razón, también funciona con listas cuyos elementos sean instancias de la clase `String`. Por ejemplo, el siguiente fragmento de código:

```
final fabricantes = ['Asus', 'Toshiba', 'Apple', 'Lenovo'];
fabricantes.sort();
print(fabricantes);
```

imprime en la consola los elementos de la lista ordenados por orden alfabético:

```
[Apple, Asus, Lenovo, Toshiba]
```

Invertir una lista

Dada una lista, la propiedad `reversed` devuelve un `Iterable` con los mismos elementos pero en **orden inverso** al que tienen en la lista original.

Por ejemplo, el siguiente fragmento de código:

```
final fabricantes = ['Asus', 'Toshiba', 'Apple', 'Lenovo'];
final fabricantesInvertida = fabricantes.reversed;
print(fabricantes);
print(fabricantesInvertida);
```

muestra en la consola de depuración:

```
[Asus, Toshiba, Apple, Lenovo]
(Lenovo, Apple, Toshiba, Asus)
```

Si se observa detenidamente el ejemplo anterior se pueden ver algunos aspectos interesantes sobre la propiedad `reversed` de la clase `List`:

- El resultado de llamar a la propiedad `reversed` es un `Iterable`. Eso se ve porque la segunda línea de la salida en la consola va escrita entre paréntesis `( )` y no entre corchetes `[ ]` como corresponde cuando lo que se imprime es una lista.
- No reordena la lista original, sino que devuelve un nuevo `Iterable` con los elementos de la lista original en orden inverso.

Importante: Una propiedad interesante de los `Iterable` es que tienen **inicialización diferida**. Eso significa, que hasta que no se usa la lista invertida por primera vez, no se realiza la inversión realmente.

Listas mutables e inmutables

En los ejemplos anteriores, las listas que se han construido se han declarado con `var` por lo que podían hacerse cosas como:

```
var fabricantes = ['Apple', 'Asus', 'Lenovo', 'Toshiba'];
fabricantes = [];
fabricantes = ['Acer', 'Dell'];
```

Si se sustituye `var` por `final`:

```
final fabricantes = ['Apple', 'Asus', 'Lenovo', 'Toshiba'];
fabricantes = []; // Error de compilación
fabricantes = ['Acer', 'Dell']; // Error de compilación
```

se obtiene el error de compilación:

```
Error: Can't assign to the final variable 'fabricantes'.
```

Declarar la lista `fabricantes` como `final` la hace constante, es decir, no se puede asignar a `fabricantes` ninguna otra lista diferente, pero ¿qué ocurre con la propia lista inicial?

Si escribimos el siguiente fragmento de código:

```
final fabricantes = ['Apple', 'Asus', 'Lenovo', 'Toshiba'];
fabricantes.remove('Lenovo');
fabricantes[1] = 'Dell';
print(fabricantes);
```

vemos que se ejecuta normalmente y en la consola se imprime:

```
[Apple, Dell, Toshiba]
```

La lista `fabricantes` no se hace inmutable al usar la palabra reservada `final`, sino que solo la variable `fabricantes` es inmutable. No se puede asignar una nueva lista a `fabricantes` pero sí se puede cambiar la lista que ya tiene asignada.

Para hacer que las listas sean **profundamente inmutables**, es decir, que no se puedan cambiar tampoco sus elementos, se tienen tres opciones:

1. La primera es usar la palabra reservada `const`. Si se escribe el siguiente fragmento de código:

```
const fabricantes = ['Apple', 'Asus', 'Lenovo', 'Toshiba'];
fabricantes.remove('Lenovo'); // Error en tiempo de ejecución
fabricantes[1] = 'Dell'; // Error en tiempo de ejecución
```

No solo la variable `fabricantes` es constante, también la lista que contiene es inmutable y por tanto no puede cambiarse.

Importante: Hay que tener en cuenta que en este caso, el error que se produce es un error **en tiempo de ejecución** y no en tiempo de compilación.

```
UnsupportedError (Unsupported operation: Cannot remove from an unmodifiable list)
```

2. La segunda opción es útil cuando no se puede hacer que la variable `fabricantes` sea `const`. En este caso se usa `const` pero no delante de la variable `fabricantes` sino delante del literal de lista:

```
final fabricantes = const ['Apple', 'Asus', 'Lenovo', 'Toshiba'];  
fabricantes.remove('Lenovo'); // Error en tiempo de ejecución  
fabricantes[1] = 'Dell'; // Error en tiempo de ejecución
```

En el fragmento de código anterior la lista es inmutable, no pueden modificarse sus elementos. Como en el caso anterior, el error que se produce es un error en tiempo de ejecución.

Esta situación se da cuando se quiere dar un valor por defecto a una propiedad de una clase que contiene una lista. En este caso, la propiedad no puede hacerse `const`, solo `final`, pero al hacerla `final` la lista en sí ya no es inmutable. La solución es usar la palabra reservada `const` delante del literal de lista.

Por ejemplo:

```
class Fabricantes {  
    final List<String> fabricantes;  
    Fabricantes([this.fabricantes = const ['Acer', 'Asus']]);  
}  
  
void main() {  
    var unObjeto = Fabricantes();  
    unObjeto.fabricantes[1] = 'Apple'; // Error en tiempo de ejecución  
}
```

En la clase `Fabricantes` se ha declarado una propiedad `fabricantes` que es `final` y se le ha asignado un valor por defecto que es inmutable. Si al construir un objeto de la clase `Fabricantes` no se especifica ningún argumento para el constructor, se toma el valor por defecto y por tanto los elementos de la lista no se pueden modificar. Se produce el error en tiempo de ejecución:

```
Unsupported operation: Cannot modify an unmodifiable list
```

Sin embargo, si al llamar al constructor, se especifica una lista no constante, sí que se pueden cambiar sus valores. Por ejemplo, el siguiente fragmento de código:

```
var unObjeto = Fabricantes(['Acer', 'Asus']);  
unObjeto.fabricantes[1] = 'Apple';  
print(unObjeto.fabricantes);
```


se ejecuta con normalidad e imprime en la consola de depuración:

```
[Acer, Apple]
```

3. La tercera opción se usa cuando se necesita una lista inmutable, pero sus valores no se conocen en tiempo de compilación, sino en tiempo de ejecución. En ese caso, como sabemos, no se puede usar la palabra reservada `const` sino que obligatoriamente hay que usar `final`.

Para resolver el problema se usa el constructor con nombre `List.unmodifiable` que devuelve una **copia** no modificable de la lista que se le pasa como argumento.

```
final listaModificable = ['Hola', 'Mundo', 'Cruel'];
final listaNoModificable = List<String>.unmodifiable(listaModificable);

listaModificable[0] = 'Adiós'; // Permitido
listaNoModificable[0] = 'Adios'; // Error en tiempo de ejecución
```

Muy Importante: Si no se invoca al constructor con nombre indicando el tipo de los elementos de la lista: `List.unmodifiable(listaModificable)`; el tipo que infiere Dart para la lista no modificable es `List<dynamic>` que es un tipo bastante peligroso porque produce errores que son difíciles de encontrar y corregir, por eso siempre se debe indicar el tipo. En este ejemplo: `List<String>.unmodifiable(listaModificable)`;

Propiedades interesantes de las listas

La clase `List` tiene muchas propiedades interesantes como por ejemplo:

```
final unaLista = ['Hola', 'Mundo', 'Cruel'];

print(unaLista.first); // Hola
print(unaLista.last); // Cruel

print(unaLista.isEmpty); // false
print(unaLista.isNotEmpty); // true

print(unaLista.length); // 3
```

Sublistas

En ocasiones no se necesitan todos los elementos de una lista, sino solo un subconjunto de ellos correlativos. En ese caso se puede usar el método `sublist` de la clase `List`.

Este método recibe dos argumentos, los índices del primer y el último elemento y **devuelve una nueva lista** que contiene los elementos cuyo índice es mayor o igual que el del primer elemento y menor estricto que el del último. Los elementos de la sublista están en el mismo orden que tienen en esta lista original.

Por ejemplo, el siguiente fragmento de código en Dart:

```
var colores = ['rojo', 'verde', 'azul', 'naranja', 'rosa'];  
print(colores.sublist(1, 3));
```

imprime en la consola:

```
[verde, azul]
```

Nota: El elemento con índice `3` no se incluye en la sublista.

Iterar sobre los elementos de una lista

En muchas ocasiones, se requiere recorrer todos los elementos de una lista para realizar alguna acción sobre ellos. Esto puede hacerse mediante bucles. En Dart se pueden usar dos bucles para iterar sobre una lista.

Nota: También podrían usarse high-order methods que se estudian más adelante en esta sección.

El primer bucle que se puede usar es el bucle `for`. Por ejemplo, si se escribe el siguiente fragmento de código:

```
final fabricantes = ['Apple', 'Asus', 'Lenovo', 'Toshiba'];  
  
for (int i=0; i<fabricantes.length; i++) {  
    print('Me gusta ${fabricantes[i]}');  
}
```

se imprime en la consola de depuración:

```
Me gusta Apple  
Me gusta Asus  
Me gusta Lenovo  
Me gusta Toshiba
```

Nota: Las llaves `{ }` que se usan en la sustitución `${fabricantes[i]}` son muy importantes, de lo contrario lo que se sustituye es la lista completa `fabricantes` y la salida en la consola no es la que se busca.

Esta forma es más difícil de leer y más propensa a equivocarse a la hora de escribirla, por eso Dart proporciona un segundo bucle mucho mejor. Es el bucle `for-in`. El siguiente fragmento de código es equivalente al anterior:

```
for (final fabricante in fabricantes) {  
    print('Me gusta $fabricante');  
}
```

El bucle `for-in` recorre en orden todos los elementos de la lista que se indica a continuación de la palabra reservada `in`. El cuerpo del bucle se ejecuta tantas veces como elementos haya en la lista. En cada paso por el cuerpo del bucle en la variable que se declara a continuación del `for` se almacena temporalmente el elemento que corresponde con ese paso.

Como puede verse es mucho más simple.

Otros operadores para listas

Los siguientes operadores son especialmente útiles cuando se programa la interfaz de usuario de una aplicación móvil con Flutter.

Operación de propagación

El **operador de propagación** `...` o spread operator, permite expandir una o varias listas dentro de otra. Es parecido a fusionar todas las listas en una sola. Por ejemplo, si se tienen las listas:

```
const unosFabricantes = ['Apple', 'Asus', 'Lenovo'];
const otrosFabricantes = ['Toshiba', 'Dell'];
```

se pueden propagar dentro de otra lista de forma muy sencilla con el operador `...`. El siguiente fragmento de código:

```
const fabricantes = [...unosFabricantes, 'HP', ...otrosFabricantes];
print(fabricantes);
```

imprime en la consola de depuración:

```
[Apple, Asus, Lenovo, HP, Toshiba, Dell]
```

Este resultado, podría conseguirse por otros medios, pero así queda mucho más claro y conciso.

Nota: Se usa mucho cuando se construyen vistas (widgets) para una interfaz de usuario en Flutter.

También existe el **operador de propagación para nulos** `...?`. Se debe usar cuando una lista pueda ser `null`. En ese caso la omite. Por ejemplo, si se tienen las listas:

```
List<String>? unosFabricantes = ['Apple', 'Asus', 'Lenovo'];
List<String>? otrosFabricantes;
```

el siguiente fragmento de código:

```
final fabricantes = [...?unosFabricantes, 'HP', ...?otrosFabricantes];
print(fabricantes);
```

se ejecuta correctamente e imprime en la consola:

```
[Apple, Asus, Lenovo, HP]
```

Nota: En este ejemplo sencillo, el análisis de flujo de Dart funciona correctamente y el primer operador de propagación para nulos (el que precede a la lista `unosFabricantes`) no es en realidad necesario. Para esa lista se podría usar el operador de propagación normal `...`

if para colecciones (collection if)

Cuando se crea una lista, se puede usar `if` para colecciones para determinar si incluir o no un determinado elemento.

Por ejemplo, si se tiene alergia a los cacahuets, se deberían eliminar los cacahuets de la lista de ingredientes.

Con la ayuda de un `if` para colecciones es muy sencillo:

```
const alergia = true;
const ingredientes = [
  'Tomate',
  'Lechuga',
  'Carne',
  if (!alergia) 'Cacahuets',
];
print(ingredientes);
```

El fragmento de código anterior imprime en la consola:

```
[Tomate, Lechuga, Carne]
```

tal y como corresponde a alguien alérgico. Si ahora la primera línea se cambia por:

```
const alergia = false;
```

el resultado que se obtiene en la consola es:

```
[Tomate, Lechuga, Carne, Cacahuets]
```

En Flutter se usa mucho para incluir o no una vista (widget) en una lista de vistas en función de si se cumple o no una condición.

Importante: Como puede verse, el uso de `if` para colecciones no impide a una lista que sea `const` (constante en tiempo de compilación), con las ventajas que eso tiene a la hora de hacer más rápida la ejecución de la aplicación.

for para colecciones (collection for)

Cuando se crea una lista se puede usar `for` para colecciones para transformar los elementos de una lista antes de añadirlos a otra.

Por ejemplo, si se tiene una lista de ingredientes escritos en minúsculas y se necesitan en mayúsculas, se puede utilizar un `for` para colecciones:

```
const ingredientes = ['Tomate', 'Lechuga', 'Carne'];
final ingredientesMayusculas = [
  'MAYONESA',
  for (var ingrediente in ingredientes) ingrediente.toUpperCase(),
];
print(ingredientesMayusculas);
```

El fragmento de código anterior imprime en la consola:

```
[MAYONESA, TOMATE, LECHUGA, CARNE]
```

Nota: En este caso, no es posible definir la variable `ingredientesMayusculas` como `const`

Listas anulables

Dart ofrece múltiples posibilidades para gestionar listas y valores nulos.

La propia lista es anulable

En ocasiones se necesitará que la propia lista pueda ser `null` pero no así los valores que contiene. En este caso se usa el símbolo de interrogación `?` justo a continuación del ángulo de cierre `>`. Por ejemplo, el siguiente fragmento de código:

```
List<int>? listaAnulable = [2, 4, 6, 8];
listaAnulable = null;
```

define una lista anulable de enteros. La propia lista puede ser `null`, pero si no lo es, entonces todos los valores que contiene son no nulos.

Los elementos de la lista son anulables

Otras veces lo que se necesita es una lista no anulable, pero que los elementos que contiene sí puedan ser `null`. En este caso se usa el símbolo de interrogación `?` justo a continuación del nombre del tipo de los elementos y dentro de los ángulos `< >`. Por ejemplo, el siguiente fragmento de código:

```
List<int?> elementosAnulables = [2, 4, null, 3, 7];
```

declara una lista que nunca puede ser `null`, cuyos elementos sí pueden serlo, ya que son de la clase `int?`

Tanto la lista como sus elementos son anulables

Por último, en Dart existe la posibilidad de que tanto la lista, como los elementos que contiene sean anulables. En este caso, se usa dos veces el símbolo de interrogación, a continuación del nombre del tipo y a continuación del ángulo de cierre `>`. Por ejemplo, el siguiente fragmento de código:

```
List<int?>? listaYElementosAnulables = [2, 4, null, 3, 7];
listaYElementosAnulables = null;
```

declara una lista que puede ser `null`, en la que algunos de sus elementos también pueden ser `null`

Uso de operadores para valores nulos en listas

Todos los operadores para valores nulos vistos anteriormente se aplican a las lista anulables. Por ejemplo, el siguiente fragmento de código:

```
List<String?>? fabricantes = ['Toshiba', 'Dell', null, 'Apple'];
for (String? fabricante in fabricantes) {
  int letras = fabricante?.length ?? 0;
  print(letras);
}
```

imprime en la consola de depuración:

```
7
4
0
5
```

Cosas a tener en cuenta en esas instrucciones:

- Aunque la lista es anulable, Dart usa el análisis de flujo para ver que en realidad tiene un valor no nulo, por eso aplica promoción de tipos y permite usarla sin ninguna comprobación de `null` adicional.
- En el ejemplo, se ha declarado la lista como de tipo `List<String?>?`, pero dado que se inicializa con un literal de lista que contiene valores que son tanto de tipo `String` como el valor `null`, también podría haberse declarado como `List?` y Dart habría inferido el tipo correcto.
- En el `for` se ha definido la condición como `(String? fabricante in fabricantes)` pero por la misma razón, se podría haber declarado como `(var fabricante in fabricantes)` con idéntico resultado.
- Ya que los elementos de la lista pueden ser `null` se usa el operador de acceso para nulos `?.` cuando se accede a la propiedad `length` y se usa el operador `if-null ??` para garantizar que todas las cadenas tienen bien calculado el número de letras. En el caso de que la cadena sea `null` se le asigna el valor `0`

Operadores para valores nulos específicos para listas

Además de los operadores para valores nulos normales, existen dos operadores específicos para manejar listas.

EL primero es el **operador índice para nulos** `?[ ]` Es muy útil cuando la propia lista puede ser `null` Por ejemplo, supongamos la siguiente lista anulable:

```
List<String>? fabricantes;
```

La lista es `null` ya que no se le ha asignado ningún valor todavía. Si se intenta acceder a algún elemento, se produce el error de compilación:

```
Error: Operator '[' cannot be called on 'List<String>?' because it is potentially null
```

Para evitarlo, se usa el operador índice para nulos:

```
String? fabricante = fabricantes?[1];  
print(fabricante);
```

que hace que se muestre en la consola de depuración:

```
null
```

El segundo es el **operador de propagación para nulos** `...?` también muy útil cuando la lista que se propaga puede ser `null`

Si de nuevo se tiene la lista anulable:

```
List<String>? fabricantes;
```

Esa lista es `null` ya que no se le ha asignado todavía valor. Si se intenta propagar dentro de otra lista, se produce el error de compilación:

```
Error: An expression whose value can be 'null' must be null-checked before it can be  
dereferenced.
```

Para evitarlo se usa el operador de propagación para nulos `...?`

```
final empresas = ['Amazon', ...?fabricantes];  
print(empresas);
```

que hace que se muestre en la consola de depuración:

```
[Amazon]
```

Importante: La lista resultado no es `[Amazon, null]` ya que `fabricantes` es en sí misma `null`, no una lista que tiene dentro el valor `null`, por tanto no hay nada que propagar.

Copiar listas

Como sabemos, **en Dart todas las variables almacenan referencias** a instancias de clases en memoria. Eso significa que cuando se asigna una variable a otra, en la segunda no queda una copia de la instancia referenciada por la primera variable, sino otra referencia.

Con las listas ocurre lo mismo. Por ejemplo, el siguiente fragmento de código:

```
final listaA = ['Hola', 'Mundo', 'Cruel'];
final listaB = listaA;

listaA[0] = 'Adios';

print(listaA);
print(listaB);
```

imprime en la consola:

```
[Adios, Mundo, Cruel]
[Adios, Mundo, Cruel]
```

ya que tanto la variable `listaA` como la variable `listaB` hacen referencia a la misma instancia de la clase `List`.

Si lo que se quiere es hacer una **copia profunda** de la lista contenida en la variable `listaA` se debe usar el método `toList` de la clase `List`. Por ejemplo, si se sustituye el fragmento de código anterior por:

```
final listaA = ['Hola', 'Mundo', 'Cruel'];
final listaB = listaA.toList();

listaA[0] = 'Adios';

print(listaA);
print(listaB);
```

en este caso, la salida en la consola es:

```
[Adios, Mundo, Cruel]
[Hola, Mundo, Cruel]
```

Conjuntos

Los conjuntos en Dart son colecciones en las que **el orden de los elementos no es importante y no se permiten elementos duplicados**. Si se añade dos veces el mismo elemento a un conjunto, solo se almacena una vez. En los conjuntos, los elementos no tienen un orden en particular, lo que importa es saber si un elemento pertenece al conjunto o no. Eso hace que sean **más rápidos que las listas**, sobre todo si se trata de conjuntos grandes con muchos elementos.

Crear un conjunto

La clase conjunto en Dart se llama `Set` y al igual que la clase `List` es genérica, es decir que se puede tener un conjunto cuyos elementos sean de cualquier tipo, solo hay que indicar el tipo entre ángulos `< >`

Por ejemplo, la siguiente instrucción crea un conjunto de `int` vacío:

```
final Set<int> unConjunto = {};
```

también se podría haber usado la siguiente notación más corta:

```
final unConjunto = <int>{};
```

En Dart se usan llaves `{ }` para delimitar el **literal de conjunto**, que en este ejemplo está vacío.

Si en tiempo de compilación se conocen los elementos del conjunto, se puede declarar el conjunto así:

```
final otroConjunto = {1, 2, 3, 1};
```

En este caso, el literal de conjunto: `{1, 2, 3, 1}` no está vacío, por lo que Dart es capaz de **inferir el tipo** de los elementos del conjunto y no ha sido necesario indicarlo.

Imprimir conjuntos

Como sabemos, en Dart cualquier colección se puede imprimir en la consola de depuración directamente. Por ejemplo si tras el fragmento de código anterior se escribe:

```
print(otroConjunto);
```

se imprime en la consola:

```
{1, 2, 3}
```

ya que el último elemento está repetido y solo se almacena una vez.

Nota: Cuando se muestra un conjunto en la consola, sus elementos se escriben entre llaves `{ }` para indicar que se trata de un conjunto.

Averiguar si un elemento está en el conjunto

Para averiguar si un conjunto contiene a un elemento, se utiliza el método `contains`, que devuelve un valor de tipo `bool`.

Por ejemplo, el siguiente fragmento de código:

```
final marcas = {'Ford', 'Peugeot', 'Toyota'};
print(marcas.contains('Peugeot'));
print(marcas.contains('Suzuki'));
```

imprime en la consola:

```
true
false
```

Nota: El método `contains` de la clase `Set` es muy rápido. Tiene una complejidad de tiempo constante **O(1)**. Eso significa que no importa cuántos elementos contenga el conjunto, `contains` tardará la misma cantidad de tiempo en averiguar si existe un valor particular en el conjunto o no. Hay algunas excepciones a esa regla, pero es cierto en el caso promedio.

Añadir nuevos elementos a un conjunto

Los conjuntos en Dart también tienen **tamaño variable**, se puede añadir y eliminar elementos a un conjunto ya creado.

Para añadir elementos se usa el método `add` de la clase `Set`. Por ejemplo, el siguiente fragmento de código en Dart:

```
final marcas = <String>{};
marcas.add('Ford');
marcas.add('Peugeot');
marcas.add('Ford');
print(marcas);
```

muestra en la consola:

```
{Ford, Peugeot}
```

El método `add` no hace nada si el elemento que se le pasa como argumento ya pertenece al conjunto.

Nota: Este método también es muy rápido. Tiene orden de complejidad **O(1)**

Añadir varios elementos a un conjunto

También se pueden añadir varios elementos a la vez al conjunto. Para ello se usa el método `addAll` de la clase `Set`. Este método recibe como argumento **una lista**.

Por ejemplo, el siguiente fragmento de código:

```
final marcas = <String>{};
marcas.addAll(['Ford', 'Peugeot', 'Toyota']);
print(marcas);
```

imprime en la consola de depuración:

```
{Ford, Peugeot, Toyota}
```

Importante: El parámetro del método `addAll` es de tipo `List`

Eliminar elementos de un conjunto

Para eliminar elementos de un conjunto se usa el método `remove` de la clase `Set`. Por ejemplo, si a continuación del código anterior se escribe:

```
marcas.remove('Peugeot');  
print(marcas);
```

en la consola se imprime:

```
{Ford, Toyota}
```

Nota: Este método también es muy rápido. Tiene orden de complejidad **O(1)**

Iterar sobre los elementos de un conjunto

Los conjuntos también son colecciones iterables, por lo que se pueden recorrer todos sus elementos usando el bucle `for-in`.

Por ejemplo, el siguiente fragmento de código:

```
final marcas = {'Ford', 'Peugeot', 'Toyota'};  
for (final marca in marcas) {  
    print(marca);  
}
```

recorre todos los elementos en el conjunto `marcas` y para cada uno de ellos ejecuta el código del cuerpo del bucle. En cada paso por el cuerpo del bucle, en la variable que se escribe a continuación de `for` se coloca el elemento correspondiente.

El resultado es que en la consola se imprime:

```
Ford  
Peugeot  
Toyota
```

Copiar conjuntos

Al igual que con cualquier otra variable en Dart, las variables declaradas de tipo `Set` almacenan **referencias** a instancias de la clase `Set`. Por tanto, asignar una variable a otra lo que hace es que ambas referencien a la misma instancia de `Set` en la memoria.

Por ejemplo, el siguiente fragmento de código en Dart:

```
final unConjunto = {1, 3, 5, 7};
final otroConjunto = unConjunto;

unConjunto.remove(3);

print(unConjunto);
print(otroConjunto);
```

hace que en la consola se imprima:

```
{1, 5, 7}
{1, 5, 7}
```

ya que tanto `unConjunto` como `otroConjunto` son referencias a la misma instancia de la clase `Set` en memoria.

Si se quiere una **copia profunda** de un conjunto, hay que usar el método `toSet` de la clase `Set`. Por ejemplo, si se sustituye el código anterior por:

```
final unConjunto = {1, 3, 5, 7};
final otroConjunto = unConjunto.toSet();

unConjunto.remove(3);

print(unConjunto);
print(otroConjunto);
```

ahora en `otroConjunto` hay una copia profunda de `unConjunto` y cuando se cambia el conjunto al que referencia `unConjunto` los cambios no afectan también a la copia. Por tanto, el resultado en la consola es:

```
{1, 5, 7}
{1, 3, 5, 7}
```

Otros operadores para conjuntos

Operador de propagación

El **operador de propagación** `...` o spread operator, permite expandir uno o varios conjuntos dentro de otro. Por ejemplo, el fragmento de código:

```
final unasMarcas = {'Ford', 'Peugeot', 'Toyota'};
final otrasMarcas = {'Citroën', 'Toyota'};
final marcas = {...unasMarcas, 'Suzuki', ...otrasMarcas};
print(marcas);
```

produce en la consola la siguiente salida:

```
{Ford, Peugeot, Toyota, Suzuki, Citroën}
```

Importante: Como es de esperar, si hay elementos repetidos, en el conjunto resultado, solo se conserva uno de ellos.

También existe el **operador de propagación para valores nulos** `...?` que al igual que ocurría en las listas permite que Dart tenga **seguridad fuerte para valores nulos** (sound null safety).

Su uso es idéntico al que se estudió para listas.

if para colecciones (collection if)

Cuando se construye un nuevo conjunto, el operador **if para colecciones** permite asignar o no nuevos elementos al conjunto en función de si se cumple o no alguna condición. Veamos un ejemplo de uso:

```
const marcasEuropeas = true;
const marcas = {
  'Ford',
  'Toyota',
  if (marcasEuropeas) 'Peugeot'};
print(marcas);
```

el fragmento de código anterior imprime en la consola:

```
{Ford, Toyota, Peugeot}
```

Importante: El uso de **if para colecciones** no impide que el conjunto se haga constante, lo que tiene ventajas a la hora del rendimiento. En este caso en el **if** también debe escribirse una condición constante, de lo contrario se recibe un error de compilación.

for para colecciones (collection for)

Este operador permite iterar sobre los elementos de un conjunto para transformarlos antes de utilizarlos en la creación de otro conjunto. Por ejemplo, el siguiente fragmento de código:

```
const marcas = {'Ford', 'Peugeot', 'Toyota'};
final marcasMinusculas = {
  for (var marca in marcas) marca.toLowerCase(),
  'suzuki'
};
print(marcasMinusculas);
```

produce en la consola la siguiente salida:

```
{ford, peugeot, toyota, suzuki}
```

Nota: En este caso, no es posible declarar `marcasMinusculas` como `const`. Si se necesita que el resultado sea `const` se puede usar el constructor con nombre de la clase `Set` `unmodifiable`:

```
final marcasMinusculasConstante = Set<String>.unmodifiable(marcasMinusculas);
```

Operaciones booleanas sobre conjuntos

Los conjuntos en Dart implementan las operaciones booleanas unión, intersección y diferencia que permiten construir nuevos conjuntos a partir de los elementos de otros dos conjuntos preexistentes.

Importante: En ninguna de las operaciones siguientes se modifican los conjuntos originales, sino que se devuelve **un nuevo conjunto** con los resultados.

Supongamos los dos siguientes conjuntos de `int`:

```
final setA = {8, 2, 3, 1, 4};  
final setB = {1, 6, 5, 4};
```

Intersección

La **intersección** de los conjuntos `setA` y `setB` es un nuevo conjunto que **contiene los elementos comunes a ambos**. Si no tienen ningún elemento en común, el resultado es el conjunto vacío.

En Dart la intersección se calcula mediante el método `intersection` de la clase `Set`. Por ejemplo, el siguiente fragmento de código:

```
final interseccion = setA.intersection(setB);  
print(interseccion);
```

produce en la consola la siguiente salida:

```
{1, 4}
```

ya que `1` y `4` son los únicos elementos presentes tanto en `setA` como en `setB`

Unión

La **unión** de los conjuntos `setA` y `setB` es un nuevo conjunto formado a partir de la **combinación de los elementos de ambos**. Como es natural, si un elemento está en ambos conjuntos, solo se incluye una vez. En Dart la unión se calcula mediante el método `union` de la clase `Set`.

Por ejemplo, el siguiente fragmento de código:

```
final union = setA.union(setB);  
print(union);
```

imprime en la consola:

```
{8, 2, 3, 1, 4, 6, 5}
```

Los elementos `4` y `1` aparecen en ambos conjuntos, sin embargo solo se han incluido en el resultado una vez.

Diferencia

La **diferencia** entre los conjuntos `setA` y `setB` es un nuevo conjunto formado por aquellos **elementos que están en `setA` y que no están en `setB`**.

En Dart, la diferencia se calcula mediante el método `difference` de la clase `Set`. Por ejemplo, el siguiente fragmento de código:

```
final diferencia = setA.difference(setB);  
print(diferencia);
```

muestra en la consola de depuración:

```
{8, 2, 3}
```

Los elementos `1` y `4` del conjunto `setA` no se incluyen ya que también están en `setB`.

Conjuntos anulables

Al igual que las listas, los conjuntos también pueden ser anulables, tener el elemento `null` o ambas cosas.

Nota: En el caso de que un conjunto tenga el elemento `null` solo lo puede tener una vez ya que en los conjuntos no hay elementos repetidos.

Para hacer un conjunto anulable, es decir, un conjunto que pueda ser `null` se usa la sintaxis:

```
Set<int>? conjuntoAnulable = {1, 2, 3};  
conjuntoAnulable = null; // Permitido, ya que el conjunto es anulable
```

Si lo que se quiere es permitir que el conjunto pueda incluir al elemento nulo `null` pero que el propio conjunto no pueda ser `null`, entonces se usa la siguiente sintaxis:

```
Set<int?> elementosAnulables = {1, 2, 3};  
elementosAnulables.add(null); // Permitido porque los elementos son anulables  
elementosAnulables = null; // No permitido porque el conjunto no es anulable
```

Y si se necesita que tanto el conjunto como sus elementos puedan ser `null` entonces se usa la siguiente sintaxis:

```
Set<int?> conjuntoElementosAnulables = {1, 2, 3};
conjuntoElementosAnulables.add(null); // Permitido porque los elementos son anulables
conjuntoElementosAnulables = null; // Permitido porque el conjunto es anulable
```

Nota: Todo lo estudiado en el apartado de listas sobre operadores seguros para nulos, se aplica también a los conjuntos de la misma forma.

Mapas

Los mapas en Dart son la estructura de datos que permite almacenar **pares llave-valor**. En otros lenguajes de programación se los conoce como HashMaps o Dictionaries.

Nota: Se podría pensar en un mapa como una colección de variables que contienen valores. la llave es el nombre de la variable y el valor, el valor de esa variable. La forma de acceder a un valor en el mapa es mediante la llave que está asociada a ese valor.

Crear un mapa

La clase a partir de la cual se construyen las instancias de los mapas es `Map` que es una clase genérica. Se usan los ángulos `< >` para indicar el tipo tanto de la llave, como de su valor asociado.

Por ejemplo, el siguiente fragmento de código:

```
final Map<String, int> unMapa = {};
```

que también se puede escribir de forma más breve como:

```
final unMapa = <String, int>{};
```

declara una variable llamada `unMapa` de la clase `Map` en el que cada par está formado por una llave de tipo `String` y un valor de tipo `int`

Aspectos importantes sobre los mapas:

- `Map` es una clase genérica, para poder usarla hay que concretar el tipo que se quiere entre ángulos `< >`, pero en este caso hacen falta dos tipos separados por comas. El primero corresponde con el tipo de la llave y el segundo con el tipo del valor asociado a esa llave.
- Al igual que en los conjuntos se usan llaves `{ }` para encerrar los elementos de los **literales de mapa**.

Eso puede ser motivo de confusión. Por ejemplo, ¿qué declara la siguiente instrucción en Dart?:

```
final algo = {};
```

La instrucción anterior es completamente válida y se ejecuta correctamente, pero podría referirse tanto a un conjunto como a un mapa. ¿Cuál de las dos infiere Dart?

La variable `algo` así definida se infiere como de tipo `Map<dynamic, dynamic>`. Como sabemos, eso puede generar bastantes problemas en tiempo de ejecución, por tanto, cuando se quiera declarar un mapa vacío siempre se deben especificar los tipos tanto de la llave como de su valor.

Si se quiere inicializar un mapa con valores, se usan **literales de mapa**. Por ejemplo, el siguiente fragmento de código:

```
final inventario = {  
  'disco duro': 4,  
  'placa base': 2,  
  'memoria': 10,  
};
```

declara una variable `inventario` de tipo `Map` con llaves de tipo `String` y valores de tipo `int` y la inicializa con tres pares. Algunas consideraciones importantes con respecto al código anterior:

- En este caso, no ha sido necesario especificar los tipos de la llave y el valor, ya que se ha usado un literal de mapa y Dart ha podido inferir los tipos a partir de ese literal.
- Un literal de mapa se escribe entre llaves `{ }`, los distintos pares que lo forman se separan entre sí por comas `,` y en cada par, la llave va separada de su valor mediante dos puntos `:`.
- La última coma (la que sigue al par `'memoria': 10`) es opcional, pero si se pone, nos aseguramos que Dart formateará el código verticalmente de forma automática.

La llave no tiene por qué ser una cadena de caracteres, ni el valor un número entero. Por ejemplo, el siguiente fragmento de código:

```
final dias = {  
  1: 'lunes',  
  2: 'martes',  
  3: 'miércoles',  
  4: 'jueves',  
  5: 'viernes',  
  6: 'sábado',  
  7: 'domingo',  
};
```

declara una variable `dias`, de tipo `Map<int, String>`, que podría usarse para convertir un número entre `1` y `7` en el día de la semana correspondiente.

Imprimir un mapa

Como cualquier otra colección, los mapas en Dart pueden imprimirse directamente. Por ejemplo, las instrucciones siguientes:

```
print(inventario);  
print(dias);
```

imprimen en la consola de depuración:

```
{disco duro: 4, placa base: 2, memoria: 10}  
{1: lunes, 2: martes, 3: miércoles, 4: jueves, 5: viernes, 6: sábado, 7: domingo}
```

Al imprimir un mapa, los pares se escriben entre llaves.

Llaves únicas

Las llaves en un mapa **son únicas**. Por ejemplo el siguiente fragmento de código:

```
final inventario = {  
  'disco duro': 4,  
  'placa base': 2,  
  'memoria': 10,  
  'disco duro': 1,  
};  
print(inventario);
```

imprime en la consola de depuración:

```
{disco duro: 1, placa base: 2, memoria: 10}
```

La llave `disco duro` se usa dos veces en el literal de mapa. la primera vez se crea el par en el mapa y se le asigna el valor `4`, la siguiente vez se sustituye el valor `4` por el nuevo, que en este ejemplo, es `1`

Nota: Como es de esperar, los valores no tienen la obligación de ser únicos. Varias llaves distintas en el mapa pueden tener asociado el mismo valor.

Acceder a los pares del mapa

Para acceder al valor asociado a un par en el mapa se usa su llave escrita entre corchetes `[ ]`. Por ejemplo, las siguientes líneas de código:

```
print(inventario['placa base']);  
print(dias[1]);
```

imprimen en la consola:

```
2  
lunes
```

Podría ser que en el mapa no haya ningún par cuya llave sea la que se pasa dentro de los corchetes `[ ]`, por esa razón el operador `[ ]` devuelve un tipo anulable. Por ejemplo, en la siguiente declaración:

```
final numeroPlacasBase = inventario['placa base'];
```

Dart infiere que el tipo de `numeroPlacasBase` es `int?` Por eso, si se quiere utilizar, hay que usar los operadores para valores nulos. Por ejemplo, el operador de acceso para nulos:

```
print(numeroPlacasBase?.isEven);
```

que imprime `true` en la consola.

Importante: Si no se usan los operadores para valores nulos, se obtienen errores en tiempo de compilación.

```
Error: Property 'isEven' cannot be accessed on 'int?' because it is potentially  
null.  
``
```

Añadir pares a un mapa

Añadir nuevos pares a un mapa ya creado en Dart es sencillo, solo hay que utilizar alguna llave que no esté aún en el mapa. Por ejemplo, el siguiente fragmento de código:

```
inventario['usb'] = 4;  
print(inventario);
```

muestra en la consola:

```
{disco duro: 1, placa base: 2, memoria: 10, usb: 4}
```

Actualizar un par en un mapa

Para actualizar el valor de un elemento que ya esté contenido en el mapa, se asigna un valor a una llave que ya exista. Por ejemplo, el siguiente fragmento de código:

```
inventario['placa base'] = 0;
```

Hace que el mapa quede:

```
{disco duro: 1, placa base: 0, memoria: 10, usb: 4}
```

Recuerda: Las llaves en los mapas **son únicas**, por lo que no se añade de nuevo el par con clave `placa base` sino que se actualiza su valor.

Eliminar pares de un mapa

Para eliminar un par de un mapa en Dart, se usa el método `remove` de la clase `Map` y se le pasa como argumento la llave del par que se quiere eliminar.

Por ejemplo, el siguiente fragmento de código:

```
inventario.remove('placa base');
```

hace que el mapa quede así:

```
{disco duro: 1, memoria: 10, usb: 4}
```

Nota: En el caso de que se pase como argumento del método `remove` una llave que no exista, no se hace nada y el mapa queda igual.

Propiedades interesantes de los mapas

La clase `Map` tiene muchas propiedades interesantes. Si se tiene el siguiente mapa:

```
final inventario = {  
  'disco duro': 4,  
  'placa base': 2,  
  'memoria': 4,  
};
```

se puede averiguar si está vacío, no vacío y su número de pares con las propiedades:

```
inventario.isEmpty // false  
inventario.isNotEmpty // true  
inventario.length // 3
```

Otras propiedades interesantes son `keys` que devuelve un `Iterable` con las llaves del mapa, y `values` que devuelve un `Iterable` con los valores del mapa. En este ejemplo, las siguientes instrucciones:

```
print(inventario.keys);  
print(inventario.values);
```

imprimen en la consola:

```
(disco duro, placa base, memoria)
(4, 2, 4)
```

Importante: En la consola se muestran paréntesis `( )` en lugar de corchetes `[ ]` o llaves `{ }` ya que el tipo que devuelven ambas propiedades es `Iterable`

Si en lugar de la clase `Iterable` se necesita que el resultado sea una lista o un conjunto se pueden usar los métodos `toList` o `toSet` de la clase `Iterable`

Por ejemplo:

```
print(inventario.keys.toList());
print(inventario.values.toSet());
```

Muestra en la consola:

```
[disco duro, placa base, memoria]
{4, 2}
```

Comprobar la existencia de una llave o un valor

La clase `Map` tiene dos métodos muy útiles que permiten averiguar si en un determinado mapa existe una determinada llave o valor. Son `containsKey` y `containsValue`.

Por ejemplo:

```
inventario.containsKey('usb') // false
inventario.containsValue(4) // true
```

Iterar sobre los pares de un mapa

A diferencia de listas y conjuntos (que son iterables), en los mapas no se puede usar el bucle `for-in` directamente para iterar sobre los pares del mapa, sino que hay que usar un `Iterable`. Recordemos que las propiedades `keys` y `values` de los mapas devuelven objetos de la clase `Iterable` que sí pueden usarse en el bucle `for-in`.

Por ejemplo, el siguiente código:

```
for (var key in inventario.keys) {
    print(inventario[key]);
}
```

imprime en la consola:

```
4  
2  
4
```

También puede usarse la propiedad `entries` de la clase `Map` que devuelve un objeto de la clase `Iterable` que contiene todos los pares del mapa. Por ejemplo:

```
for (var entry in inventario.entries) {  
    print('${entry.key} -> ${entry.value}');  
}
```

muestra en la consola de depuración:

```
disco duro -> 4  
placa base -> 2  
memoria -> 4
```

Copiar mapas

Al igual que con cualquier otra variable en Dart, las variables declaradas de tipo `Map` almacenan **referencias** a instancias de la clase `Map`. Por tanto, asignar una variable a otra, lo que hace es que ambas referencien a la misma instancia de `Map` en la memoria.

Por ejemplo, el siguiente fragmento de código en Dart:

```
final inventario = {  
    'disco duro': 4,  
    'placa base': 2,  
    'memoria': 4,  
};  
final otroInventario = inventario;  
otroInventario['memoria'] = 30;  
print(inventario);  
print(otroInventario);
```

muestra en la consola:

```
{disco duro: 4, placa base: 2, memoria: 30}  
{disco duro: 4, placa base: 2, memoria: 30}
```

como vemos, el valor de la llave `memoria` se ha modificado tanto en el mapa referenciado por la variable `inventario` como en el mapa referenciado por `otroInventario` ya que en realidad son el mismo mapa.

Si se quiere una **copia profunda** de un mapa, se puede usar el método `from` de la clase `Map`. Por ejemplo, se puede usar la asignación:

```
final otroInventario = Map.from(inventario);
```

En este caso, la variable `otroInventario` referencia a un mapa distinto pero con los mismos pares que `inventario`. Es una copia de `inventario` por lo que los cambios que se hagan en uno no afectan al otro.

¿Cuándo usar cada colección

Cada tipo de colección tiene ventajas e inconvenientes. En este resumen hay algunos consejos sobre cuándo usar cada una de ellas:

- **Listas:** Se usan las listas `List` si **el orden es importante**. En este caso, lo mejor es insertar elementos al final de la lista siempre que sea posible para maximizar la velocidad, sobre todo en listas con gran número de elementos. Por lo general, las búsquedas son lentas cuando el número de elementos es elevado.
- **Conjuntos:** Se usan los conjuntos `Set` si **el orden no importa** y solo interesa si un elemento pertenece al conjunto o no. Las búsquedas en conjuntos son mucho más rápidas que en listas.
- **Mapas:** Se usan los mapas `Map` si es necesario poder **buscar valores mediante una llave**. La búsqueda mediante una llave también es una operación más rápida que la búsqueda en listas.

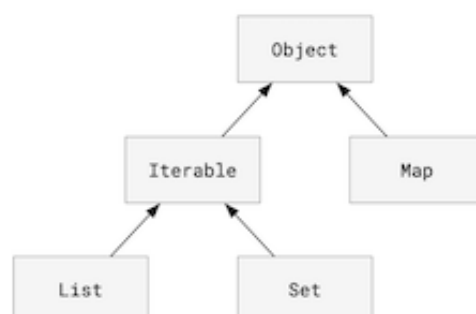
Higher order methods

Cuando estudiamos las funciones, aprendimos que las funciones que devuelven a otras funciones como resultado, o que las reciben como argumentos, se denominan **higher order functions**. Son fundamentales en el paradigma de programación llamado **programación funcional**.

Dart es un lenguaje de programación orientado a objetos, pero también permite utilizar programación funcional lo que le da una gran potencia y flexibilidad.

Las colecciones en Dart tienen **higher order methods** que permiten realizar operaciones complejas sobre la colección de forma muy sencilla y eficiente.

Importante: Los **higher order methods** que se van a estudiar pertenecen a la clase `Iterable`. Tanto las listas `List` como los conjuntos `Set` son iterables, por tanto, esos métodos se les pueden aplicar directamente. Los mapas `Map` por el contrario, no son iterables y para poder aplicarles higher order methods hay que usar las propiedades `keys` o `entries` que devuelven un `Iterable`. Sobre los iterables devueltos sí se pueden aplicar los higher order methods.



Iterar sobre una colección

Además de los bucles `for-in` que ya hemos visto, las colecciones en Dart tienen el método `forEach` que es un higher order method, es decir, recibe como argumento una función anónima. `forEach` recorre cada uno de los elementos de la colección y ejecuta sobre él, la función anónima que se le pasa.

La función anónima que se pasa al método `forEach` recibe un argumento del mismo tipo que los elementos en la colección.

Veamos al método `forEach` en acción. El siguiente fragmento de código:

```
const numeros = [1, 2, 3];
numeros.forEach((int numero) {
  print(numero * 3);
});
```

muestra en la consola la siguiente salida:

```
3
6
9
```

Aspectos importantes sobre el método `forEach` :

- Recorre cada uno de los elementos de la colección. En este ejemplo, la colección es una lista con tres elementos, por tanto se itera tres veces. En cada iteración, se usa uno de los elementos de la colección.
- La función anónima que se pasa como argumento se aplica una vez en cada iteración. El resultado es que a cada elemento se le aplica una vez la función anónima.

La sintaxis del método `forEach` puede parecer confusa al principio con tantos paréntesis y llaves mezclados. Para aclararla un poco, vamos a escribir los dos componentes por separado.

- `forEach` es un método que se aplica a una colección. La sintaxis es la de siempre para cualquier método:

```
numeros.forEach(  
  
);
```

Se escribe la variable que contiene al objeto sobre el que se quiere aplicar `forEach` seguido de un punto `.` a continuación el nombre del método, los argumentos entre paréntesis `()` y punto y coma `;` al final.

- El argumento que hay que pasar a `forEach` es una función anónima con un parámetro de tipo `int` ya que los elementos de la lista del ejemplo son de este tipo. Si la colección tuviera otros elementos de otro tipo, ese sería el tipo que habría que poner al argumento de la función anónima.

La sintaxis de una función anónima es la que ya hemos estudiado:

```
(int numero) {  
    print(numero * 3);  
}
```

Cuando se escribe todo junto queda como en el ejemplo del principio.

Dado que los elementos de la lista, en este ejemplo, son de tipo `int` Dart puede inferir el tipo del parámetro de la función anónima. Por ejemplo, también se podría haber escrito de esta forma:

```
const numeros = [1, 2, 3];  
numeros.forEach((numero) {  
    print(numero * 3);  
});
```

sin explicitar el tipo del parámetro, ya que Dart puede inferirlo a partir del tipo de los elementos de la colección.

También se puede escribir usando la notación flecha para la función anónima ya que, en este ejemplo, el cuerpo de la función anónima solo tiene una línea. Quedaría así:

```
const numeros = [1, 2, 3];  
numeros.forEach((numero) => print(numero * 3));
```

lo que es una notación más compacta. Las tres versiones son equivalente, puede usarse cualquiera de ellas indistintamente.

Importante: El `;` que sigue a la instrucción `print` hay que quitarlo cuando se usa notación flecha o se obtiene un error en tiempo de compilación:

```
Error: Expected ')' before this.
```

Las funciones anónimas se pueden guardar en variables, así que lo anterior también podría haberse codificado así:

```
const numeros = [1, 2, 3];  
final triplicar = (numero) => print(numero * 3);  
numeros.forEach(triplicar);
```

Que posiblemente sea la forma más clara y sencilla de entender.

Nota: Toda esta notación y formas distintas de escribir el high order method `forEach` se aplica a los demás high order methods, por tanto en las siguientes secciones no se va a repetir.

Iterar sobre un mapa

En el caso de los mapas, el uso de `forEach` es ligeramente distinto. La diferencia es que la clase `Map` define su propio higher order method `forEach` y en él la función anónima que se pasa no debe tener un parámetro sino dos. Uno con el tipo de la llave del mapa y otro con el tipo del valor del mapa.

Por ejemplo:

```
final inventario = {  
  'disco duro': 4,  
  'placa base': 2,  
  'memoria': 4,  
};  
  
inventario.forEach((llave, valor) {  
  print('Hay $valor unidades de $llave');  
});
```

El resultado impreso en la consola de ese fragmento de código es:

```
Hay 4 unidades de disco duro  
Hay 2 unidades de placa base  
Hay 4 unidades de memoria
```

Transformar los elementos de una colección

Las colecciones tienen un higher order method método llamado `map` que recibe como argumento una función anónima y **produce un nuevo `Iterable`** resultado de aplicar la función anónima a cada elemento de la colección sobre la que se llama el método `map`.

Por ejemplo, el siguiente fragmento de código:

```
const numeros = [1, 2, 3, 4];  
final cuadrados = numeros.map((numero) => numero * numero);  
print(cuadrados);
```

construye un `Iterable` llamado `cuadrados` que contiene el resultado de elevar al cuadrado el valor de los cuatro elementos de la lista `numeros`

El resultado que se muestra en la consola es:

```
(1, 4, 9, 16)
```

En este ejemplo, el tipo del parámetro de la función anónima no se ha escrito explícitamente ya que Dart puede inferirlo directamente a partir de los valores que hay en la lista `numeros`. También se ha usado la notación flecha más compacta ya que para elevar al cuadrado solo hace falta una línea de código.

Nota: `map` no produce como resultado una colección del mismo tipo que la colección sobre la que se aplica, sino que siempre produce un `Iterable`. La razón es el rendimiento. Recuerda que los `Iterable` tienen **inicialización diferida**, es decir, no se inicializan realmente hasta la primera vez que se necesitan.

Recuerda: Como ya se vio anteriormente, la clase `Iterable` tiene métodos `toList` y `toSet` para transformar el `Iterable` a otros formatos cuando sea necesario.

Importante: No hay que confundir el método `map` de las colecciones que produce un `Iterable` con la colección `Map` que almacena pares formados por una llave y un valor. Su nombre es muy parecido pero `map` es un método de la clase `Iterable` mientras que `Map` es una clase.

Filtrar una colección

Se pueden filtrar los elementos de una colección que cumplan una determinada condición y construir con ellos un nuevo `Iterable`. Para ello se usa el método `where` de las colecciones.

El método `where` es un high order method, recibe como argumento una función anónima que tiene un parámetro del mismo tipo que los elementos de la colección y que devuelve un valor booleano indicando si ese elemento cumple o no la condición. El higher order method `where` usa ese valor booleano devuelto para añadir (`true`) o no (`false`) el elemento en el `Iterable` resultado.

Por ejemplo, si se tiene una lista de enteros y se quiere filtrar para obtener un `Iterable` con solo aquellos que sean impares se puede usar `where` de la siguiente forma:

```
final numeros = [1, 2, 3, 4, 5, 6];
final impares = numeros.where((numero) => numero.isOdd);
print(impares);
```

El resultado en la consola es:

```
(1, 3, 5)
```

Resumir una colección

Algunos higher order methods toman los elementos de una colección y la resumen en **un único valor** usando para ello la función anónima que se pasa como argumento. La función anónima se aplica una vez para cada elemento de la colección.

Hay dos versiones: `reduce` y `fold`

reduce

Con frecuencia se tiene un array de valores y se necesita calcular su suma. Se trata de resumir los elementos de la colección en uno solo cuyo valor es la suma de los demás. Este es un ejemplo claro en el que se puede usar el método `reduce` de las colecciones.

El siguiente fragmento de código:

```
const numerosReales = [2.3, 8.4, 3.6, 8.0, 10.2, 4.1];
final suma = numerosReales.reduce((sumaParcial, numero) => sumaParcial + numero);
print(suma);
```

calcula la suma de los elementos de la lista y muestra en la consola:

```
36.6
```

Algunas cosas a tener en cuenta sobre el código del ejemplo:

- La función anónima, tiene dos parámetros. El primero es siempre el resultado obtenido en la llamada anterior a la función (recuerda que la función anónima se llama una vez para cada elemento de la colección). El segundo argumento contiene el elemento que se está recorriendo en esa llamada.
- En el método `reduce` el primer parámetro se inicializa con el primer valor de la colección. En este ejemplo, `sumaParcial` comienza valiendo `2.3`. A continuación se recorren el resto de elementos de la lista aplicando para cada uno la función anónima y el resultado se guarda en el primer argumento, en este caso en `sumaParcial`. De esta forma, `sumaParcial` va acumulando el resultado hasta esa iteración.
- Cuando se termina de recorrer todos los elementos de la colección, el método `reduce` devuelve el valor que haya en el primer argumento de la función anónima, en este ejemplo en `sumaParcial`.

El siguiente ejemplo de código usando el método `reduce` no funciona:

```
final listaVacia = <double>[];
final suma = listaVacia.reduce((sumaParcial, elemento) => sumaParcial + elemento);
print(suma);
```

Se produce un error en tiempo de ejecución ya que al estar la lista vacía, no es posible inicializar el primer argumento, en este ejemplo `sumaParcial`, con el primer elemento de la lista. El error que se muestra en la consola es:

```
Bad state: No element
```

El método `resume` solo puede aplicarse si la lista tiene uno o más elementos. El método `fold` resuelve este problema.

fold

En el método `fold` además de la función anónima, se recibe otro argumento que es el valor de comienzo. La función anónima sigue teniendo dos parámetros, pero ahora el primero de ellos no se inicializa con el primer elemento de la colección, sino con el nuevo argumento que se pasa a la función `fold`.

Veamos por ejemplo, como se calcularía la suma de los elementos de una lista, pero ahora usando el método `fold` :

```
const numerosReales = [2.3, 8.4, 3.6, 8.0, 10.2, 4.1];
final suma = numerosReales.fold(0.0, (sumaParcial, numero) => sumaParcial + numero);
```

- Como puede verse, `fold` tiene dos argumentos. El primero es el valor inicial para el parámetro donde se va acumulando el resultado. En este ejemplo, como lo que se calcula es la suma de los elementos de la lista, se ha elegido el valor `0.0` que es el elemento neutro de la suma y no altera el resultado final. En otros casos habrá que escoger el valor inicial apropiado a ese caso.
- La función anónima es la misma que se usó en el ejemplo con el método `reduce` ya que el cálculo es el mismo. La única diferencia entre `reduce` y `fold` es de dónde se toma el valor inicial para el valor acumulado.

No hay problema en usar el método `fold` sobre una colección vacía, ya que el valor inicial se da en un argumento. Por ejemplo, el código que fallaba cuando se usaba `reduce` sí funciona bien cuando se usa `fold` :

```
final listaVacía = <double>[];
final suma = listaVacía.fold(0.0, (sumaParcial, elemento) => sumaParcial + elemento);
print(suma);
```

en la consola de depuración se muestra:

```
0
```

Ordenar listas de forma personalizada

El método `sort` de las listas tiene un **parámetro opcional** que es una **función anónima**, por lo tanto es un higher order method. El valor por defecto es `null` pero si se pasa una función anónima en ese argumento, se usa para establecer el orden de los elementos de la lista.

La función anónima que se pase debe tener dos parámetros del mismo tipo que los elementos de la lista (los dos elementos a comparar) y devolver `-1` si el primer elemento va antes que el segundo, `1` si el segundo va antes que el primero y `0` si ambos elementos tienen el mismo orden.

Por ejemplo, si tenemos una lista de cadenas de caracteres podemos cambiar el comportamiento por defecto del método `sort` que es ordenarlas por orden alfabético, pasando nuestra propia función anónima.

En el siguiente fragmento de código, se usa una función anónima para ordenar las cadenas según su longitud. Primero van las cadenas con menos caracteres.

```
var fabricantes = ['Toshiba', 'Dell', 'hp', 'Apple'];
fabricantes.sort((f1, f2) => f1.length.compareTo(f2.length));
print(fabricantes);
```

en la consola se imprime:

```
[hp, Dell, Apple, Toshiba]
```

Cosas a tener en cuenta:

- `compareTo` es un método de la clase `num` que compara dos números y devuelve `-1` si el primero es menor que el segundo, `0` si son iguales y `1` si el segundo es mayor que el primero.

Aquí se ha usado para comparar la longitud de dos cadenas de caracteres de una forma más compacta y sencilla de escribir.

- Como en otros higher order methods, el tipo de `f1` y `f2` no se ha explicitado, ya que como se aplica a la lista `fabricantes` que es una lista de `String`, Dart puede inferir el tipo sin problemas.

Mini ejercicios

1. Crea una lista de `String` vacía con nombre `meses`. Usa el método `add` para rellenarla con los nombres de los doce meses del año.
2. Construye una lista inmutable con los mismos valores del ejercicio anterior.
3. Usa `collection-for` para crear otra nueva lista con los nombres de los meses en mayúsculas.
4. Construye un mapa con las siguientes llaves: `nombre`, `profesion`, `pais` y `ciudad`. Para los valores asociados usa tus propios datos.
5. Si decides mudarte a Toronto en Canadá, qué órdenes te permitirían actualizar el mapa para reflejar esa nueva situación.
6. Itera sobre el mapa para imprimir todos los valores.
7. Construye una lista que contenga todos los valores del mapa.
8. 🔄 Escribe una función llamada `diferenciaSimetrica` que reciba como argumentos dos conjuntos y devuelva otro conjunto que sea la diferencia simétrica de los dos.

Nota: La diferencia simétrica de dos conjuntos es el conjunto formado por todos los elementos que están solo en uno de los dos.

Por ejemplo si se tienen los conjuntos:

```
final setA = {8, 2, 3, 1, 4};  
final setB = {1, 6, 5, 4};
```

el resultado debería ser el conjunto:

```
{8, 2, 3, 6, 5}
```

No se incluye ni el `1` ni el `4` porque están en ambos conjuntos.

9. Se tiene una colección en memoria, al imprimirla en la consola se muestra:

```
{1, 4, 9, 16}
```

¿De qué clase de colección se trata? Justifica tu respuesta.

10. He usado la siguiente línea de código para consolidar todos los valores de la lista que se encuentra en la variable `cantidades` :

```
final total = cantidades.reduce((suma, elemento) => suma + elemento);
```

y se ha producido un error en tiempo de ejecución. ¿Qué ha podido pasar? Explica con detalle tu respuesta.

11. Se tiene una colección en memoria, al imprimirla en la consola se muestra:

```
(1, 4, 9, 16)
```

¿De qué clase de colección se trata? Justifica tu respuesta.

12. Dada la siguiente lista de notas en un examen:

```
final notas = [8.9, 7.7, 4.6, 9.3, 8.2, 6.7, 3.2, 8.8];
```

- Usa el método `sort` de la clase `List` para encontrar los valores máximo y mínimo. Explica cómo lo has hecho.
- Usa el higher order method `where` para localizar todos los notables, es decir, las notas que están dentro del intervalo `[7,9)`

13. 🕒 Se tiene la siguiente lista:

```
var lista = ['Ford', 'Range Rover', 'Peugeot', 'Toyota', 'DS', 'BMW'];
```

Escribe el código más simple que puedas para encontrar la cadena de menor longitud y la de mayor longitud.

14. 🕒 Escribe una función en Dart que reciba como argumento una lista y devuelva un booleano indicando si la lista tiene elementos duplicados (`true`) o no (`false`).

15. ➔ Escribe una función llamada `enterosAleatorios`, con un parámetro con nombre llamado `digitos` que cumpla lo siguiente:
- $3 \leq \text{digitos} \leq 10$
 - la función devuelve una lista de enteros, en la que haya tantos enteros como se indique en `digitos`
 - Los enteros de la lista solo pueden pertenecer al conjunto de dígitos del sistema de numeración decimal: `{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}`
 - No se pueden repetir elementos en la lista resultado, es decir, no puede aparecer dos o más veces un mismo elemento del conjunto de dígitos del sistema de numeración decimal.
 - Los elementos de la lista deben aparecer en orden aleatorio.

Ejemplos de salidas de esta función con cinco dígitos pueden ser:

```
[9, 8, 6, 0, 5]
[6, 3, 2, 4, 7]
[3, 5, 0, 4, 2]
```

y con tres dígitos:

```
[1, 7, 4]
[3, 7, 4]
[6, 0, 8]
```

Escribe la función usando el menor número de líneas de código que te sea posible.

Pista: Busca en la documentación oficial de Dart (<https://api.dart.dev/>) información sobre el método `shuffle()` de la clase `List` para averiguar cómo funciona e intenta usarlo en tu código.

16. Escribe una clase `Usuario` con dos propiedades `id` y `nombre`. A continuación construye una lista con tres instancias de la clase `Usuario` usando los `id` y `nombre` que prefieras. Por último, escribe una función que convierta la lista de usuarios en un mapa cuyos pares tengan como llave `id` y como valor `name`.

Dart 8: Uso avanzado de clases

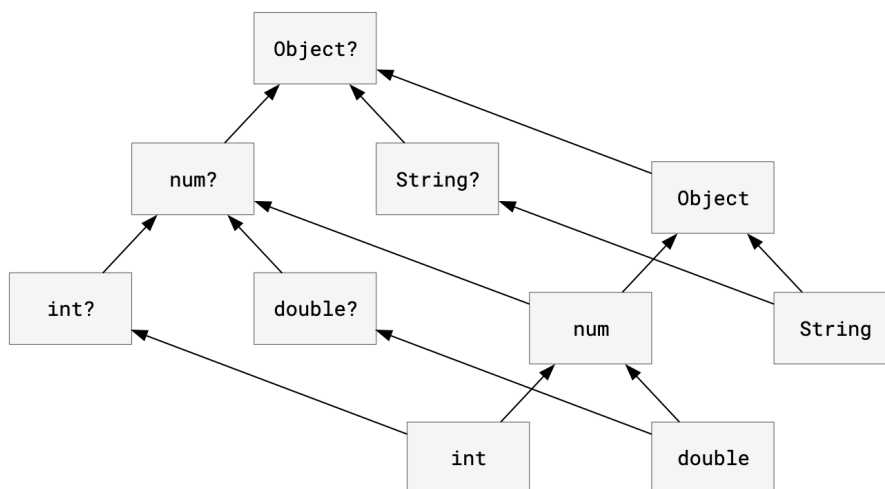
- Dart 8: Uso avanzado de clases
 - Jerarquías de clases
 - Declarar subclases
 - Forma corta del constructor
 - Añadir propiedades a una subclase
 - super y listas de inicialización
 - Redefinir métodos de la superclase
 - Llamar a super desde un método redefinido
 - Clases hermanas
 - Inferencia de tipos en una lista mezclada
 - Comprobando el tipo en tiempo de ejecución
 - Clases abstractas
 - ¿Cuándo declarar métodos concretos y cuándo declarar métodos abstractos?
 - Interfaces
 - Utilidad de las interfaces
 - Declaración de la interface
 - Implementación de la interface
 - Uso de la interface
 - Extensiones
 - Ejemplo de extensión
 - Tipos genéricos
 - Crear una clase genérica propia
 - Usar una clase genérica propia
 - Añadir propiedades a una clase genérica
 - Añadir métodos a una clase genérica
 - Tipos enumerados mejorados
 - Añadir propiedades a un tipo enumerado
 - Añadir métodos al tipo enumerado
 - Mini ejercicios

Jerarquías de clases

Una de las principales aportaciones de las **clases** a la programación es que permiten definir **jerarquías de clases** en las que una **subclase** puede extender a una **superclase**. Las propiedades y métodos de la superclase **se heredan** en las subclases.

Ya hemos trabajado con una parte de la jerarquía de clases que proporciona el lenguaje Dart por defecto. Sabemos que todas las clases no anulables en Dart son subclases de una única clase llamada `Object` y que una de las subclases de `Object` es `num` que a su vez tiene otras dos subclases llamadas `int` y `double`. Por otro lado, hemos trabajado con la clase `String` que es otra subclase de `Object`.

También sabemos que cada clase en Dart tiene una superclase anulable que tiene el mismo nombre seguido del símbolo `?`. Podríamos dibujar este fragmento de la jerarquía de clases de Dart así:



Así, la clase `int` tiene como superclase a `num`, pero también a `int?`. Sin embargo, la clase `int?` solo tiene como superclase a `num?`.

A partir de un único nodo, `Object` o bien `Object?` se construye un árbol con las distintas clases. Organizar las clases en jerarquías tiene grandes ventajas, como estudiaremos a continuación.

En esta sección aprenderemos a construir nuestras propias jerarquías de clases y lo que es más importante, en la próxima sesión de prácticas aprenderemos a usarlas para programar **patrones de diseño de software** esenciales como el patrón **estrategia**.

Consejo: Seguramente ya has visto jerarquías de clases en otros lenguajes de programación, sin embargo, Dart incorpora algunos mecanismos que es posible que desconozcas. Estos mecanismos son **fundamentales** si se quiere hacer buenas aplicaciones móviles en Flutter, así que te recomiendo que estudies con detenimiento lo que sigue a continuación.

Para los ejemplos de esta sección, definiremos la siguiente clase sencilla:

```
class Spaceship {
  int tripulacion;
  String nombre;

  Spaceship(this.tripulacion, this.nombre);

  @override
  String toString() {
    return '$runtimeType(nombre: $nombre, tripulación: $tripulacion)';
  }
}
```

Declarar subclases

En Dart se declara una subclase que **extiende** o **especializa** a una superclase mediante la palabra reservada `extends` que se usa de la siguiente forma:

```
class Subclase extends Superclase {
  // - Código de la subclase aquí
}
```

Por ejemplo, se puede definir una clase `FighterSpaceship` que especializa a la clase `Spaceship` de la siguiente forma:

```
class FighterSpaceship extends Spaceship {
  FighterSpaceship(int tripulacion, String nombre) :
    super(tripulacion, nombre);
}
```

Algunas consideraciones sobre el código anterior:

- La palabra reservada `extends` indica que la clase `FighterSpaceship` extiende o especializa a la clase `Spaceship`, que es su superclase. Esto implica que todos los métodos y propiedades de `Spaceship` se heredan en la subclase `FighterSpaceship`.
- Los constructores de la superclase **no se heredan** en la subclase, sino que deben declararse.
- En este ejemplo el constructor de la subclase no tiene cuerpo, se usa el reenvío de constructores para llamar al constructor de la superclase para que inicialice las propiedades.
- Cuando se usa en una subclase la palabra reservada `super` hace referencia a su **superclase inmediata** (la que está un nivel por encima en la jerarquía de clases). En el constructor de `FighterSpaceship` se usa para llamar al constructor de la superclase `Spaceship` y que éste inicialice las propiedades.

Nota: La palabra reservada `this` se refiere a la propia clase mientras que la palabra reservada `super` hace referencia a su superclase inmediata.

Tanto `Spaceship` como `FighterSpaceship` son clases completamente operativas. Se pueden usar de la forma habitual. Por ejemplo:

```
final nostromo = Spaceship(7, 'Nostromo');
print(nostromo);
final rodgerYoung = FighterSpaceship(325, 'Rodger Young');
print(rodgerYoung);
```

que imprime en la consola de depuración:

```
Spaceship(nombre: Nostromo, tripulación: 7)
FighterSpaceship(nombre: Rodger Young, tripulación: 325)
```

Nota: La subclase `FighterSpaceship` no tiene definido el método `toString`, sin embargo, al imprimir un objeto de esta clase no se usa el comportamiento por defecto de `toString` que hubiese impreso en la consola:

```
Instance of 'FighterSpaceship'
```

Eso es porque `FighterSpaceship` hereda todos los métodos de su superclase `Spaceship`, incluido el método `toString` redefinido. Además, en ese método se usa `$runtimeType` que en cada clase concreta devuelve su nombre.

Forma corta del constructor

En lugar de utilizar manualmente el reenvío de constructores, se podría haber escrito el constructor de la subclase usando la forma corta, así:

```
FighterSpaceship(super.tripulacion, super.nombre);
```

Ahora ya no se está usando el reenvío de constructores, sino configurando directamente los parámetros en la superclase.

Añadir propiedades a una subclase

Las subclases especializan a las superclases, es decir, siguen teniendo las mismas funcionalidades, pero de forma más concreta o específica y pueden añadir alguna característica nueva. Por ejemplo, en una nave espacial de combate, lo normal es que haya armamento. Eso se puede gestionar en la subclase `FighterSpaceship` de esta forma:

```
class FighterSpaceship extends Spaceship {
    Map<String, int> armamento;
    FighterSpaceship(super.tripulacion, super.nombre, this.armamento);
}
```

Como se puede ver, en el constructor de `FighterSpaceship` se usa la palabra reservada `super` para inicializar las propiedades heredadas de `Spaceship` y la palabra reservada `this` para inicializar sus propiedades específicas.

super y listas de inicialización

Como sabemos, cuando en una clase hay propiedades privadas, no se puede usar la forma corta del constructor, sino que en su lugar se usa una **lista de inicialización**. En ese caso, el reenvío de constructores con la palabra reservada `super` debe escribirse siempre al **final**, justo a continuación de la lista de inicialización.

Por ejemplo, si la propiedad `armamento` se quiere hacer privada, entonces no se puede usar la forma corta para escribir el constructor de `FighterSpaceship`, en su lugar hay que escribirlo así:

```
class FighterSpaceship extends Spaceship {
  Map<String, int> _armamento;
  FighterSpaceship(int tripulacion, String nombre, Map<String, int> armamento):
    _armamento = armamento,
    super(tripulacion, nombre);
}
```

Nota: Si en la superclase no hubiera constructor generativo personalizado, sino que se usa el constructor por defecto, cuando se escribe la lista de inicialización no es necesario escribir `super()` al final, porque Dart siempre llama al constructor por defecto de la superclase automáticamente. De esta forma puede asegurarse de que todos los valores de todas las propiedades se han inicializado correctamente antes de usar las instancias.

Redefinir métodos de la superclase

Tal y como está declarada en este momento la clase `FighterSpaceship` si se ejecuta el siguiente fragmento de código:

```
final rodgerYoung = FighterSpaceship(325, 'Rodger Young', {
  'Cañones plasma': 10,
  'Misiles': 24,
});
print(rodgerYoung);
```

se imprime en la consola:

```
FighterSpaceship(nombre: Rodger Young, tripulación: 325})
```

Como vemos no se imprime el armamento. Esto es así, porque se usa el método `toString` de `Spaceship` y dado que esa clase no tiene la propiedad `_armamento`, su método `toString` no tiene en cuenta esa propiedad.

En casos como este, se puede redefinir cualquier método de la superclase **sustituyendo** su funcionalidad para adaptarla a las necesidades de la subclase. La forma de redefinir un método en Dart es usando exactamente **la misma signatura** del método que se quiere redefinir precedido de la anotación `@override`, seguido del cuerpo con la nueva implementación del método que se necesite.

En este ejemplo se sobrescribe el método `toString` de esta forma:

```
@override
String toString() {
    return '$runtimeType(nombre: $nombre, tripulación: $tripulacion, ' +
        'armamento: $_armamento)';
}
```

Una vez redefinido el método, si se vuelve a ejecutar el fragmento de código:

```
print(rodgerYoung);
```

se obtiene en la consola la salida correcta:

```
FighterSpaceship(nombre: Rodger Young, tripulación: 325, armamento: {Cañones plasma: 10,
Misiles: 24})
```

Nota: La anotación `@override` se estudió brevemente en la primera sesión de prácticas. Técnicamente el uso de `@override` es opcional en Dart, pero ayuda mucho a evitar errores, ya que si se usa y por error no se escribe correctamente la signatura del método a redefinir, el compilador produce un error avisando de que el método en realidad no existe en la superclase.

Llamar a super desde un método redefinido

Otras veces no se redefine un método de la superclase porque se quiera **sustituir** su funcionalidad, sino porque se quiere **añadir** nueva funcionalidad a la que ya tiene.

En ese caso, como parte del cuerpo del método redefinido, se puede hacer una llamada con `super` al método original de la superclase que se está redefiniendo.

Tal y como está en este momento nuestra jerarquía de clases, tenemos dos clases operativas, pero con poca funcionalidad. Una cosa que cualquier buen comandante de una nave espacial debe hacer es realizar una inspección prevuelo para garantizar que todo está en perfecto estado de funcionamiento.

Podemos añadir esta funcionalidad a la superclase `Spaceship` añadiendo el siguiente método:

```
void inspeccionPrevuelo() {
    print('Comprobando motores.');
```

```
    print('Comprobando soporte vital.');
```

```
}
```

Este método se hereda por la subclase, así que si se ejecuta el siguiente código:

```
final rodgerYoung = FighterSpaceship(325, 'Rodger Young', {
    'Cañones plasma': 10,
    'Misiles': 24,
});
rodgerYoung.inspeccionPrevuelo();
```

se obtiene en la consola:

```
Comprobando motores.  
Comprobando soporte vital.
```

Lo cual es correcto, pero solo en parte. En una nave espacial de combate, además de la comprobación de motores y el soporte vital, se debería comprobar también el armamento. No se trata de sustituir completamente el comportamiento del método `inspeccionPrevuelo` sino de añadirle nueva funcionalidad.

Para ello se redefine el método `inspeccionPrevuelo` en la subclase `FighterSpaceship` con una llamada a `super.inspeccionPrevuelo()` y las nuevas tareas de inspección:

```
@override  
void inspeccionPrevuelo() {  
    super.inspeccionPrevuelo();  
    print('Comprobando armamento:');  
    for (var arma in _armamento.keys) {  
        print(arma);  
    }  
}
```

Cuando se prueba la nueva clase con la instrucción:

```
rodgerYoung.inspeccionPrevuelo();
```

se obtiene en la consola:

```
Comprobando motores.  
Comprobando soporte vital.  
Comprobando armamento:  
Cañones plasma  
Misiles
```

Es decir, se ejecutan **las dos versiones** del método `inspeccionPrevuelo`, la de la superclase y la de la subclase.

Nota: En este ejemplo, la llamada al método de la superclase se ha colocado al principio del cuerpo del método redefinido, pero puede colocarse en cualquier lugar donde sea necesario.

Clases hermanas

Otra clase especial de nave espacial son las naves de exploración. Lo que distingue a esta clase de naves de otras naves espaciales es que dispone de varios sensores de exploración. Los sensores son los que especializan a esa clase de naves.

Podríamos definir una clase `ExplorationSpaceship` para encapsular el comportamiento de esta clase de naves. Podría ser algo así:

```
class ExplorationSpaceship extends Spaceship {
    Map<String, int> _sensores;

    ExplorationSpaceship(int tripulacion, String nombre, Map<String, int> sensores):
        _sensores = sensores,
        super(tripulacion, nombre);

    @override
    void inspeccionPrevuelo() {
        super.inspeccionPrevuelo();
        print('Comprobando sensores:');
        for (var sensor in _sensores.keys) {
            print(sensor);
        }
    }

    @override
    String toString() {
        return '$runtimeType(nombre: $nombre, tripulación: $tripulacion, ' +
            'sensores: $_sensores)';
    }
}
```

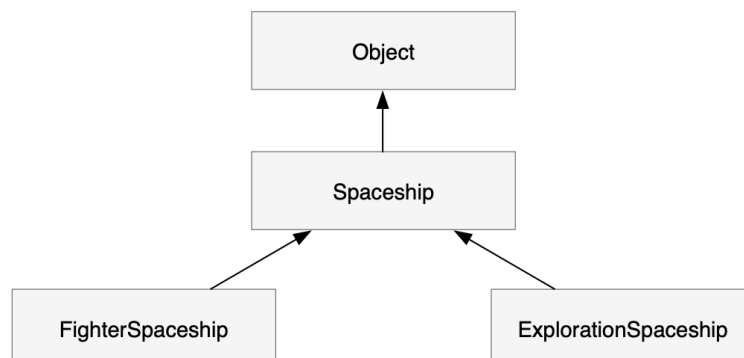
El uso de esta clase es el habitual. Por ejemplo, el siguiente fragmento de código:

```
eventHorizon = ExplorationSpaceship(18, 'Event horizon', {
    'Sensores atmosféricos': 4,
    'Sensores partículas subatómicas': 16,
    'Sensores formas vida': 2,
});
print(eventHorizon);
```

escribe en la consola de depuración:

```
ExplorationSpaceship(nombre: Event horizon, tripulación: 18, sensores: {Sensores
atmosféricos: 4, Sensores partículas subatómicas: 16, Sensores formas vida: 2})
```

Tanto `FighterSpaceship` como `ExplorationSpaceship` extienden a la misma superclase `Spaceship`, por eso se denominan **clases hermanas**. Eso permite mucha flexibilidad a la hora de programar. El diagrama de este fragmento de la jerarquía de clases es:



El siguiente fragmento de código:

```
Spaceship nave;

nave = FighterSpaceship(325, 'Rodger Young', {
    'Cañones plasma': 10,
    'Misiles': 24,
});
print(nave);
nave = ExplorationSpaceship(18, 'Event horizon', {
    'Sensores atmosféricos': 4,
    'Sensores partículas subatómicas': 16,
    'Sensores formas vida': 2,
});
print(nave);
```

compila y ejecuta correctamente y muestra en la consola de depuración:

```
FighterSpaceship(nombre: Rodger Young, tripulación: 325, armamento: {Cañones plasma: 10,
Misiles: 24})
ExplorationSpaceship(nombre: Event horizon, tripulación: 18, sensores: {Sensores
atmosféricos: 4, Sensores partículas subatómicas: 16, Sensores formas vida: 2})
```

Veamos algunas consideraciones sobre lo anterior:

- La variable `nave` se ha declarado como de tipo `Spaceship`. Dado que `Spaceship` es superclase tanto de `FighterSpaceship` como de `ExplorationSpaceship`, a la variable `nave` se le pueden asignar indistintamente instancias de ambas clases.
- Cuando la variable `nave` contiene una instancia de `FighterSpaceship` y se invoca alguno de sus métodos, por ejemplo: `toString`, se usa la versión de `toString` de `FighterSpaceship`. De la misma forma, cuando contiene una instancia de `ExplorationSpaceship` la versión del método que se usa es la de esta clase.
- Se está usando la variable `nave` como contenedor que puede albergar instancias de cualquiera de las subclases. Cuando contiene una instancia de alguna subclase concreta, los versiones de los métodos que se emplean, son los de esa subclase.

Este mecanismo es muy útil en programación orientada a objetos. Permite entre otras cosas programar el patrón de diseño de software llamado **estrategia** que estudiaremos en la siguiente sesión de prácticas.

Nota: Como sabemos, la capacidad de una variable de albergar indistintamente a objetos de clases hermanas se la conoce como **polimorfismo**. El polimorfismo es una parte clave de la programación orientada a objetos. En la sección denominada **Clases Abstractas** estudiaremos formas más sofisticadas de declarar clases polimórficas.

Inferencia de tipos en una lista mezclada

Una de las ventajas del polimorfismo puede verse en el siguiente ejemplo de código:

```
final naves = [rodgerYoung, eventHorizon];
```

que se compila y ejecuta sin errores. En él se ha definido una lista `naves` y se ha inicializado con un literal de lista que contiene objetos de las clases hermanas `FighterSpaceship` y `ExplorationSpaceship`

Si hacemos clic con el ratón sobre la variable `naves` en DartPad, vemos que en la zona de documentación aparece que el tipo que Dart ha inferido para `naves` es:

```
List<Spaceship>
```

Documentation

List<Spaceship> naves

local variable

Es decir, Dart usa como tipo de los elementos de la lista el **antepasado común más próximo** a todos ellos y eso nos permite mezclar objetos de clases hermanas dentro de la lista.

Nota: La clase `Object` también es antepasado común para `FighterSpaceship` y `ExplorationSpaceship` ya que `Spaceship`, la superclase de ambas, extienden a `Object`, pero no es el más próximo. Por debajo de `Object` y por encima de ambas clases está `Spaceship` y es por eso que Dart la elige como tipo para los elementos de la lista. (Véase el diagrama de la jerarquía de clases para las naves).

Comprobando el tipo en tiempo de ejecución

Como ya se ha estudiado, se puede usar la palabra reservada `is` para comprobar si un objeto dado está o no dentro de la **jerarquía** de una clase. También se puede usar `is!` que en este caso nada tiene que ver con valores nulos, sino que es la negación de `is`.

Para entender bien cómo funciona, adivina sin programarlo, el resultado de las siguientes instrucciones:

```
final rodgerYoung = FighterSpaceship(325, 'Rodger Young', {  
  'Cañones plasma': 10,  
  'Misiles': 24,  
});
```

```
print(rodgerYoung is Object);  
print(rodgerYoung is Spaceship);  
print(rodgerYoung is FighterSpaceship);  
print(rodgerYoung is! ExplorationSpaceship);
```

Ahora escribe esas instrucciones en DartPad y ejecútalas. Deberías obtener la siguiente salida:

```
true  
true  
true  
true
```

¿Coincide con tu respuesta? Veamos algunas consideraciones sobre lo anterior:

- En la declaración de la clase `Spaceship` no se usa la palabra reservada `extends` por lo que Dart automáticamente la hace subclase de `Object`. Recuerda que en Dart **todas** las clases son subclases de `Object`, bien directamente, bien indirectamente.

`FighterSpaceship` extiende a `Spaceship` por lo que también tiene a la clase `Object` en su jerarquía, es decir, que `FighterSpaceship` también es subclase de `Object`

Por eso, la primera línea de la salida en la consola es `true`

- `FighterSpaceship` extiende a `Spaceship` por tanto es su superclase directa, es decir, `Spaceship` está en la jerarquía de clases de `FighterSpaceship` así que la segunda línea de la salida es también `true`
- Por supuesto la variable `rodgerYoung` se ha declarado de tipo `FighterSpaceship` así que claramente, la tercera línea de la salida es también `true`
- Por último, `ExplorationSpaceship` es hermana de `FighterSpaceship` pero no están en la misma rama de la jerarquía de clases, es decir, ninguna extiende a la otra. Como se pregunta con la palabra reservada `is!`, la respuesta es de nuevo `true`

Nota: El signo de exclamación `!` que se escribe al final de la palabra reservada `is` no tiene nada que ver con el null-assertion operador que se estudió para valores nulos. En este contexto simplemente significa que no, es la negación.

Clases abstractas

Todas las clases que se han usado hasta ahora son **clases concretas**. Eso significa que se pueden instanciar para construir objetos a partir de ellas. En Dart también se pueden definir **clases abstractas**. Son clases de las que **no se pueden construir instancias**. Estas clases sirven para definir jerarquías de clases en las que las subclases son clases concretas.

Para declarar una clase como abstracta, se escribe la palabra reservada `abstract` antes de la palabra reservada `class`

Veamos un ejemplo. Imaginemos que estamos trabajando en un juego para móviles, en el que unos personajes pueden tener diferentes habilidades, que se caracterizan por tener un modo particular de moverse y de atacar a sus oponentes. A medida que avanza el juego, el personaje va a ir cambiando de habilidad.

En ese contexto se puede definir la clase `HabilidadPersonaje` de la siguiente forma:

```
abstract class HabilidadPersonaje {  
    void mover();  
    void atacar();  
  
    @override  
    String toString() => return 'Soy una $runtimeType';  
}
```

Veamos algunos puntos importantes sobre este código:

- La clase `HabilidadPersonaje` es abstracta ya que se ha usado la palabra reservada `abstract` en su declaración. Eso significa que **no se puede instanciar**. No tiene sentido generar objetos de esta clase. Lo que sí es útil es definir subclases con las distintas habilidades concretas que se necesiten en el juego y construir instancias de ellas.
- Los métodos `mover` y `atacar` no tienen cuerpo (código entre llaves `{ }`) sino solo signatura. A esta clase de métodos se les denomina **métodos abstractos**. Para poder usarlos **deben** redefinirse en las subclases.

Nota: En otros lenguajes de programación a esta clase de métodos se les llama **virtuales puros**.

- Una clases abstracta puede tener también métodos concretos además de los abstractos, es decir, aquellos en los que se proporciona una implementación. En este ejemplo, `toString` es un método concreto.
- El conjunto de métodos públicos de una clase abstracta define la **API común** que van a tener todas las subclases. En este ejemplo concreto, todas las subclases de `HabilidadPersonaje` van a tener los métodos `mover`, `atacar` y `toString`

Recuerda: Una **API**, o Application Programming Interface, es el conjunto de métodos públicos que permiten que un módulo de programa se comunique con otro. Hasta ahora, solo hemos usado las APIs de otros desarrolladores. Por ejemplo, hemos usado la API que proporciona el SDK de Dart cada vez que hemos usado alguno de los métodos de las clases de Dart como `String` o `List`.

Ahora ya tenemos las herramientas necesarias para desarrollar nuestras propias APIs.

Como es natural, la clase `HabilidadPersonaje` no se puede instanciar ya que es abstracta. Si se ejecuta el siguiente fragmento de código:

```
var habilidadPersonaje = HabilidadPersonaje();
```

se produce el error de compilación:

```
Error: The class 'HabilidadPersonaje' is abstract and can't be instantiated.
```

A partir de la clase `HabilidadPersonaje` se podrían declarar dos subclases que implementen dos habilidades concretas. Por ejemplo:

```
class HabilidadBasica extends HabilidadPersonaje {
    @override
    void atacar() => print('Ataco con fuerza 1');

    @override
    void mover() => print('Me muevo 1 metro');
}
```

y

```
class HabilidadSuprema extends HabilidadPersonaje {
    @override
    void atacar() => print('Ataco con fuerza 10');

    @override
    void mover() => print('Me muevo 10 metros');
}
```

Con esa jerarquía de clases se tiene mucha flexibilidad a la hora de programar el juego. Se comienza definiendo una variable `habilidadPersonaje` así:

```
HabilidadPersonaje habilidadPersonaje;
```

Al declarar la variable `habilidadPersonaje` como de tipo `HabilidadPersonaje`, por el polimorfismo, podrá contener instancias tanto de `HabilidadBasica` como de `HabilidadSuprema`

Al comienzo del juego se puede incializar con la habilidad más básica:

```
habilidadPersonaje = HabilidadBasica();
```

cuando se use:

```
print(habilidadPersonaje);
habilidadPersonaje.mover();
habilidadPersonaje.atacar();
```

se imprimirá en la consola:

```
Soy una HabilidadPersonaje
Me muevo 1 metro
Ataco con fuerza 1
```

A medida que el juego progrese, se podrá cambiar de habilidad. Reflejar ese hecho en código es trivial, solo hay que escribir:

```
habilidadPersonaje = HabilidadSuprema();
```

y ahora cuando se use la habilidad:

```
print(habilidadPersonaje);  
habilidadPersonaje.mover();  
habilidadPersonaje.atacar();
```

la salida que se obtiene en la consola será:

```
Soy una HabilidadSuprema  
Me muevo 10 metros  
Ataco con fuerza 10
```

Nota: Si se hace clic en DartPad sobre el nombre de la variable `habilidadPersonaje` en la zona de documentación se muestra que el tipo inferido es `HabilidadPersonaje`, el de la superclase, con independencia de la clase que tenga la instancia concreta que haya en `habilidadPersonaje`

Sin embargo `runtimeType` no muestra el tipo de la variable `habilidadPersonaje` sino de la instancia concreta que contiene.

¿Cuándo declarar métodos concretos y cuándo declarar métodos abstractos?

El criterio para elegir es:

- Si el comportamiento de un método de la clase abstracta va a ser diferente en cada subclase concreta, lo mejor es usar un método abstracto, lo que **obliga** a redefinirlo en todas las subclases.
- Si el comportamiento del método va a ser el mismo en cada subclase concreta, lo mejor es usar un método concreto, con lo que se encapsula su funcionalidad en la superclase en aplicación del principio de diseño de software **DRY** Don't Repeat Yourself.

Interfaces

Las interfaces son similares a las clases abstractas, ya que permiten definir el comportamiento que deben tener todas las clases que implementen esa interface. Son un medio para **ocultar los detalles de implementación** de las clases concretas del código que usa esas clases.

En Dart no existe una palabra reservada específica para indicar que una clase es una interface, sino que se usan clases abstractas para definir interfaces. Lo que diferencia si una clase abstracta actúa como clase abstracta propiamente dicha o como una interface es la forma en que se usa en las subclases.

- Si al declarar una subclase se usa la palabra reservada `extends`, la superclase se trata como una clase abstracta.

- Si al declarar una subclase se usa la palabra reservada `implements`, la superclase se trata como una interface.

A efectos prácticos, ¿cuál es la diferencia entre usar `extends` o `implements` a la hora de definir subclases? Básicamente dos:

- Una subclase solo puede extender a una única superclase, en Dart no se permite **herencia múltiple**, mientras que puede implementar tantas interfaces como sea necesario.
- Si se usa `implements`, es decir la superclase abstracta actúa como interface, la subclase debe implementar **todos** los métodos y todas las propiedades de la superclase, incluso aquellos que no sean abstractos. Es decir, las clases concretas **no heredan** ni los métodos ni las propiedades de la interface.

Por el contrario, si se usa `extends` la subclase hereda todos los métodos concretos y las propiedades de la superclase. Puede redefinirlos si lo necesita, pero no es obligatorio.

Por eso, la mayoría de interfaces se construyen a partir de clases abstractas que solo contienen métodos abstractos o virtuales puros.

Utilidad de las interfaces

Las interfaces son muy útiles en multitud de situaciones. Por ejemplo, es muy frecuente que en el momento de implementar la interfaz de usuario de una aplicación todavía no se tenga claro cómo se va a implementar la lógica de negocio de la aplicación o bien sí se tenga claro pero todavía no se haya hecho.

En estos casos, las interfaces permiten implementar y probar la interfaz de usuario de la aplicación y dejar para después el diseño e implementación de la lógica de negocio, **sin que ello obligue a rehacer el código de la interfaz de usuario** que accede a las clases de negocio. Con el consiguiente ahorro de tiempo y dinero.

Supongamos que se está implementando una aplicación para gestionar información meteorológica en la que se debe mostrar la temperatura en distintas ciudades. Ese dato se va a obtener de un repositorio externo que aún no se ha diseñado y/o implementado.

En este contexto se puede definir el repositorio como una interface de la que luego se pueden realizar diferentes implementaciones concretas.

Declaración de la interface

En este ejemplo sencillo la interface podría programarse así:

```
abstract class RepositorioDatosMeteo {  
    double? temperatura({required String ciudad});  
}
```

Como podemos ver, la declaración de una interface es idéntica a la de una clase abstracta. Define una API de comunicación formada por varios métodos abstractos públicos que **deberán implementarse** en las clases concretas que implementen esa interface.

Si se cambia la implementación de alguno de los métodos en alguna clase concreta, eso no afecta para nada a las clases usuarias de esa API, ya que la signatura de los métodos públicos no cambia.

Nota: En este ejemplo, el valor devuelto por el método `temperatura` es `double?` es decir, un valor que puede ser `null`. De esta forma se tiene en cuenta la situación en la que no se tengan datos de temperatura para la ciudad que se le pasa como argumento.

Implementación de la interface

Las clases abstractas no pueden instanciarse, así que hasta que no se declaren clases concretas que implementen esa interface, no es posible utilizarla. En este ejemplo concreto, aún no se sabe bien cómo se va a implementar la lógica de negocio de la aplicación, pero no es problema ya que puede implementarse una clase concreta que simule el comportamiento que debe tener la interface.

Por ejemplo, se puede declarar una clase que devuelva algún valor aleatoriamente para simular el comportamiento real de esa interface:

```
import 'dart:math';

class RepositorioSimulado implements RepositorioDatosMeteo {
  @override
  double? temperatura({required String ciudad}) {
    final generadorAleatorio = Random();
    final temperaturaSimulada = (generadorAleatorio.nextDouble() * 11.0) + 15.0;
    if (temperaturaSimulada > 25.0) {
      return null;
    } else {
      return temperaturaSimulada;
    }
  }
}
```

En este ejemplo concreto se generan temperaturas aleatoriamente en el intervalo `[15.0, 25.0]` o bien el valor `null`. Para ello se siguen los siguientes pasos:

1. Se usa un generador de números aleatorios al que se le pide un valor real en el intervalo `[0.0, 1.0]` mediante el método `nextDouble`.
2. A continuación se escala ese valor devuelto al intervalo `[0.0, 11.0]`
3. Después se le suma el valor `15.0` para obtener un valor en el intervalo `[15.0, 26.0]`
4. Por último, si el valor es mayor que `25.0` se devuelve `null` para simular que en algunas de las ocasiones se introduzca una ciudad para la que no hay datos disponibles.

Cuando la interfaz de usuario utilice ese repositorio simulado va a tener todo el repertorio de posibles respuestas con las que probar los casos de interés. Todo ello aunque no se haya codificado todavía el repositorio real que se va a usar.

Uso de la interface

Un posible uso de la interface `RepositorioDatosMeteo` es:

```
final RepositorioDatosMeteo repositorioTemp = RepositorioDatosMeteo();
final temperatura = repositorioTemp.temperatura(ciudad: 'Madrid');
print(temperatura);
```

Veamos algunas consideraciones sobre ese código:

- `RepositorioSimulado` es una subclase de `RepositorioDatosMeteo` por lo que, usando polimorfismo, se puede declarar una variable `repositorioTemp` de tipo `RepositorioDatosMeteo` y asignarle una instancia de la clase concreta `RepositorioSimulado`
- A partir de ahí, las clases usuarias acceden a los datos mediante los métodos públicos declarados en la interface `RepositorioDatosMeteo`, por lo que se podría cambiar el comportamiento interno del método `temperatura` anterior sin que por ello haya que cambiar nada en las clases usuarias.

Se ha reducido mucho el acoplamiento entre las clases usuarias y las clases concretas de la interface `RepositorioDatosMeteo` pero no se ha eliminado del todo. En la inicialización:

```
final RepositorioDatosMeteo repositorioTemp = RepositorioSimulado();
```

las clases usuarias usan el nombre de una clase concreta en el momento de inicializar el repositorio mediante su constructor. Cuando se tenga la implementación definitiva del repositorio de datos meteorológicos y se sustituya con ella a la implementación simulada, habrá que cambiar en las clases usuarias todas las inicializaciones como la anterior que se tengan.

Eso, como sabemos, no es deseable. Lo mejor es que un cambio en la clase concreta que se necesite no afecte en ninguna forma a las clases usuarias.

Para lograr ese desacoplamiento total se usan los constructores factoría. En la clase abstracta que define la interface se añade un constructor factoría de forma que la interface queda así:

```
abstract class RepositorioDatosMeteo {
    factory RepositorioDatosMeteo() => RepositorioSimulado();
    double? temperatura({required String ciudad});
}
```

y el uso que se hace de la interface ahora es:

```
final repositorioTemp = RepositorioDatosMeteo();
final temperatura = repositorioTemp.temperatura(ciudad: 'Madrid');
print(temperatura);
```

Veamos algunos aspectos interesantes de lo anterior:

- Se ha añadido un **constructor factoría sin nombre** a la clase abstracta `RepositorioDatosMeteo` que define la interface. Los constructores no se heredan en las subclases, por lo que no influye para nada en las implementaciones concretas de la interface.

- Ese constructor factoría devuelve un objeto de una de sus subclases concretas, lo que está permitido por el lenguaje Dart. En este caso, de la subclase concreta `RepositorioSimulado`
- Lo realmente inteligente aquí, es que, dado que se trata de un constructor factoría sin nombre, da la sensación de que se está instanciando realmente la interface abstracta `RepositorioDatosMeteo`, por lo que es muy sencillo de recordar y utilizar. Sin embargo la clase que se instancia realmente es `RepositorioSimulado`. Para demostrarlo, se puede ejecutar a continuación del código anterior:

```
print(repositorioTemp);
```

que imprime en la consola:

```
Instance of 'RepositorioSimulado'
```

- Cuando se completen otras subclases concretas que implementen el repositorio de otras formas, para probarlas, solo hay que cambiar la instancia devuelta en ese constructor factoría. Un único cambio, en un único sitio y en la jerarquía de clases que definen la interface, no en sus clases usuarias. La encapsulación es total.

Nota: Estos son solo algunos usos de las interfaces. Las interfaces también pueden usarse para implementar patrones de diseño de software muy útiles como el patrón **Estrategia** que se estudia en la siguiente sesión de prácticas.

Extensiones

Dart tiene otra característica muy potente que se llama **extensiones**. Permite añadir funcionalidad a cualquier clase existente, incluso a clases que pertenecen a una biblioteca de Dart o de terceros, de la que no se tiene el código fuente. Esas clases no se pueden modificar directamente, pero con las extensiones se les puede añadir funcionalidad.

Nota: No se trata de hacer una subclase que extienda a la clase de Dart o de terceros y en esa subclase añadir la funcionalidad que se necesite. Se trata de añadir la funcionalidad directamente a la propia clase.

Para definir una extensión se usa la siguiente sintaxis:

```
extension on UnaClase {  
    // código que añade funcionalidad a la clase  
}
```

`UnaClase` debe sustituirse por el nombre de la clase a la que se quiera añadir funcionalidad.

Importante: Ese código debe escribirse fuera de cualquier clase o función. **Las extensiones no pueden declararse dentro de ningún otro ámbito.**

Ejemplo de extensión

Imaginemos que en una aplicación concreta, se requiere calcular muchas veces y en muchas clases el cubo de un número entero. La clase `int` no dispone de un método o propiedad `cubed` o similar, por lo que se podría escribir ese cálculo así:

```
final valor = 5;
final resultado = valor * valor * valor; // 125
```

Si se debe escribir muchas veces en muchos sitios, acaba siendo complejo y propenso a fallos.

Una forma de solucionarlo es con una top-level function. Por ejemplo se puede definir la función:

```
int elevarAlCubo(int valor) => valor * valor * valor;
```

que se usaría así:

```
final valor = 5;
final resultado = elevarAlCubo(valor); // 125
```

Sin embargo, la solución mejor es usando una extensión. Se puede añadir un getter directamente a la clase `int` para tener la funcionalidad de elevar un entero al cubo. La forma de hacerlo es escribir fuera de cualquier ámbito:

```
extension on int {
  int get cubo => this * this * this;
}
```

Importante: La sintaxis que se ha empleado es la que usaríamos para definir un getter dentro de la clase `int` si tuviéramos acceso a su código fuente. Por eso se usa la palabra reservada `this` que representa al valor entero sobre el que se aplica el getter.

Un uso posible de esta extensión es:

```
print(5.cubo);
```

que muestra en la consola:

```
125
```

Como puede verse, el getter `cubo` se ha aplicado directamente a la instancia `5` de la clase `int` porque ese código de la extensión, a partir de ese momento ya forma parte de la implementación de `int`.

Por supuesto también se podría haber usado así:

```
final valor = 5;
final resultado = valor.cubo; // 125
```

Importante: Las extensiones son muy potentes, pero añaden un comportamiento no estándar a las clases, lo que puede dificultar que otros desarrolladores entiendan el código que escribamos. Cuando se usan extensiones en módulos que van a usar terceras personas es conveniente documentar bien ese hecho para facilitar su uso y comprensión.

Tipos genéricos

En Dart, la palabra **genérico** hace referencia a la capacidad de generalizar tipos concretos distintos de forma que se manejen todos de forma similar.

Ya hemos manejado tipos genéricos cuando estudiamos las colecciones ya que todas las colecciones en Dart son genéricas. Por ejemplo, al definir una lista en Dart escribíamos el tipo concreto de los elementos de la lista entre ángulos `< >`:

```
List<String> fabricantes = [];
```

El fragmento de código anterior declara una lista llamada `fabricantes` que solo podrá contener elementos de tipo `String`. Así por ejemplo, el siguiente fragmento de código:

```
fabricantes.add('Apple');
```

se ejecuta con normalidad y añade un nuevo elemento a la lista `fabricantes`. Por el contrario, el siguiente código:

```
fabricantes.add(1);
```

produce el siguiente error de compilación:

```
Error: The argument type 'int' can't be assigned to the parameter type 'String'.
```

ya que `1` no es una instancia de la clase `String`

Las colecciones en Dart se declaran como genéricas. Eso significa que pueden almacenar elementos de cualquier tipo. Eso elimina la necesidad de crear diferentes clases como por ejemplo `IntList`, `DoubleList` o `BoolList` para cada tipo para el que se necesite una lista. **Los genéricos evitan la duplicación de código.**

En este apartado vamos a aprender a declarar nuestras propias clases y métodos genéricos, de forma que puedan funcionar con elementos de cualquier tipo sin necesidad de declarar una clase para cada tipo concreto.

Crear una clase genérica propia

Como ejemplo, vamos a construir una colección de tipo FIFO o cola, ya que Dart no la proporciona y es útil en determinados contextos.

Las colas utilizan el orden FIFO (primero en entrar primero en salir), lo que significa que el primer

elemento que se añade a la cola es siempre el primero en ser eliminado. Las colas son útiles cuando se necesita mantener el orden de llegada de los elementos. En estas colecciones se suelen tener las propiedades:

- `isEmpty` que devuelve `true` si la cola está vacía y `false` en caso contrario.
- `peek` que devuelve el elemento que está el primero, pero sin sacarlo de la cola.

también se suelen tener los métodos:

- `enqueue` que sirve para poner en cola un nuevo elemento, es decir, para añadir un nuevo elemento a la cola por el final.
- `dequeue` que sirve para sacar de la cola el primer elemento.

Podremos necesitar colas cuyos elementos sean de diferentes tipos, así que lo mejor es declarar la colección genérica.

Dart proporciona un conjunto de estructuras de datos altamente optimizadas que se puede utilizar para crear nuevas estructuras de nivel superior. Una de ellas es `List`, una colección genérica que almacena una colección contigua y ordenada de elementos. Vamos a utilizar la lista `List` para crear la clase `Queue` cola.

```
class Queue<T> {  
    final _elements = <T>[];  
}
```

- El primer paso es declarar la clase `Queue` como genérica. Para ello se usa la sintaxis ya conocida, añadiendo a continuación del nombre de la clase, el tipo genérico `T` entre ángulos `< >`. Los ángulos indican que esta es una clase donde se usa un tipo genérico.

Importante: La letra `T` representa a cualquier tipo. No hay que usar la letra `T` obligatoriamente, pero es habitual usar letras mayúsculas cuando se especifica un tipo genérico. Otra letra que se también emplea mucho es: `E`

- A continuación se declara una variable privada `_elements` de tipo `List` en la que sus elementos son justamente del tipo genérico `T` asociado a la clase `Queue`

Recuerda: `final _elements = <T>[];` es equivalente a `final List<T> _elements = [];`

Esta clase es muy básica pero, como sabemos, puede usarse directamente.

Usar una clase genérica propia

Las clases genéricas propias se usan de la misma forma que se usan las clases genéricas de Dart, como `List` o `Set`. El siguiente fragmento de código:

```
final cola = Queue<int>();
```

declara una constante `cola` de tipo cola de enteros.

Importante: Para instanciar una clase genérica, hay que escribir a continuación del nombre de la clase y entre ángulos `< >` el tipo concreto que se quiera usar en la clase genérica. En este caso `int` lo que hace que la cola declarada solo pueda contener elementos enteros.

Añadir propiedades a una clase genérica

En una clase genérica, como en cualquier otra clase, es bueno definir propiedades que permitan averiguar información sobre la clase. Se definen de la forma habitual.

Por ejemplo, las propiedades para averiguar si la cola está vacía y para obtener el primer elemento, pero sin sacarlo de la cola, se pueden declarar así:

```
bool get isEmpty => _elements.isEmpty;
T? get peek => (isEmpty) ? null : _elements.first;
```

solo hay que escribir el anterior fragmento de código al final de la clase `Queue`

- La propiedad `peek` devuelve el primer elemento de la cola. Como la cola es genérica, se usa la letra `T` para denotar ese hecho.

Importante: Cada vez que se haga referencia al tipo genérico, hay que usar la misma letra que se haya usado al declarar la clase. Si se hubiera usado otra letra, como por ejemplo `E` aquí también habría que escribir esa letra.

Nota: Cuando el valor devuelto pueda ser `null` como es el caso de la propiedad `peek` cuando la cola está vacía, se usa `T?` que, como sabemos, es la versión anulable de `T`

- En la programación de ambas propiedades se usan métodos de la clase `List` que es la que estamos utilizando para implementar la clase `Queue` Eso hace que nuestra colección sea muy eficiente.
- Esas propiedades se podrían haber implementado como métodos. Por ejemplo:

```
T? peek() => (isEmpty) ? null : _elements.first;
```

sin embargo, ese código es más ineficiente y hace que las llamadas sean más ilegibles. Por eso es preferible usar propiedades para implementar la consulta de información sobre la colección y usar métodos cuando se programen procesos sobre la colección.

El uso de estas propiedades es el habitual. Por ejemplo, se puede escribir en la función `main` lo siguiente:

```
final cola = Queue<int>();
print(cola.isEmpty);
print(cola.peek);
```

y en la consola se obtiene el resultado:

```
true  
null
```

Añadir métodos a una clase genérica

Los métodos de una clase genérica también se declaran de la forma habitual. La diferencia es que si algún parámetro o tipo devuelto es genérico hay que usar la letra `T`. Por ejemplo, los métodos para añadir y sacar elementos de la cola se pueden declarar así:

```
void enqueue(T element) {  
    _elements.add(element);  
}  
  
T? dequeue() {  
    return (isEmpty) ? null : _elements.removeAt(0);  
}
```

solo hay que escribir el anterior fragmento de código al final de la clase `Queue`

- De nuevo se usan los métodos propios de la clase `List` para implementar los métodos de la clase `Queue` lo que facilita enormemente el trabajo y proporciona una implementación muy eficiente.
- Siempre que un parámetro sea del tipo genérico, o que un valor devuelto sea del tipo genérico se usa la letra `T`

Por último, se puede redeclarar el método `toString()` para imprimir la cola de forma sencilla. Una posible forma de hacerlo es añadir el siguiente fragmento de código al final de la clase `Queue`:

```
@Override  
String toString() {  
    return 'Inicio -> $_elements <- Fin';  
}
```

Con todo ello, ya se tiene una colección de tipo FIFO plenamente funcional y genérica. Se puede usar con cualquier tipo de elementos.

Por ejemplo, se podría escribir el siguiente fragmento de código:

```
final cola = Queue<int>();  
cola.enqueue(3);  
cola.enqueue(8);  
print(cola);  
final elementoInicial = cola.dequeue();  
print(elementoInicial);  
print(cola);
```

que imprimiría en la consola:

```
Inicio -> [3, 8] <- Fin
3
Inicio -> [8] <- Fin
```

Tipos enumerados mejorados

En la sesión anterior de prácticas aprendimos a usar **tipos enumerados** en Dart. Por ejemplo, para codificar un tipo enumerado básico se usaba la siguiente sintaxis:

```
enum ColorSemaforo {
  verde,
  amarillo,
  rojo,
}
```

Para usar el tipo enumerado anterior se podría usar el código siguiente:

```
final color = ColorSemaforo.amarillo;
switch(color) {
  case ColorSemaforo.verde:
    print('Avanzar');
    break;
  case ColorSemaforo.amarillo:
    print('Decelerar');
    break;
  case ColorSemaforo.rojo:
    print('Detenerse');
    break;
}
```

que produce la siguiente salida en la consola:

```
Decelerar
```

Los tipos enumerados tienen muchas ventajas frente a las constantes globales:

- A las constantes del tipo enumerado se accede a través del nombre del tipo. Por ejemplo: `ColorSemaforo.amarillo` lo que es bueno porque:
 - Permite el acceso a esas constantes desde cualquier módulo de la aplicación de una forma sencilla.
 - El nombre del tipo enumerado define el **espacio de nombres** para las constantes de ese tipo, con lo que se podrían tener otros tipos enumerados con constantes que se llamasen igual sin que por ello haya ambigüedad.
- El compilador de Dart conoce el tipo con lo que nos puede ayudar en situaciones comunes como:
 - No haber escrito una cláusula `case` para alguna constante del tipo en un `switch`
 - Haber escrito con alguna errata alguna de las constantes del tipo.

Los tipos enumerados, tienen las ventajas de las constantes de tipo `String` que son más fáciles de leer y entender; y las ventajas de las constantes de tipo `int` que producen menos errores cuando se usan para codificar.

Nota: Todos los enum extienden automáticamente la clase `Enum` de Dart. También están sellados, lo que significa que no se pueden generar subclases a partir de ellos ni mediante `extends` ni mediante `implements` y tampoco se pueden declarar instancias de ellos explícitamente.

Ese es el uso básico de los enums, sin embargo, a partir de la versión 2.17 de Dart se pueden tratar los tipos enumerados como si fueran clases, es decir, **se les puede añadir propiedades y métodos**. Eso hace que los enums en Dart sean muy potentes.

Añadir propiedades a un tipo enumerado

Una de las cosas más útiles que se puede hacer con un tipo enumerado es añadirle propiedades. Por ejemplo, se puede añadir una propiedad de tipo `String` para asociar el mensaje, directamente al tipo enumerado:

```
enum ColorSemaforo {  
  verde('Avanzar'),  
  amarillo('Decelerar'),  
  rojo('Detenerse');  
  
  final String mensaje;  
  const ColorSemaforo(this.mensaje);  
}
```

Lo que simplifica enormemente su uso. El siguiente fragmento de código es equivalente al `switch` anterior y mucho más sencillo de leer y entender:

```
final color = ColorSemaforo.amarillo;  
print(color.mensaje);
```

En la consola se imprime el mismo mensaje:

```
Decelerar
```

Algunas cosas importantes a tener en cuenta con respecto al código anterior:

- Las constantes del tipo enumerado se declaran mediante el constructor. Se escribe el nombre de la constante y a continuación, entre paréntesis, los argumentos que tenga el constructor. En este ejemplo, el mensaje asociado a cada constante.
- **El constructor debe declararse como `const`** De lo contrario se produce el error de compilación:

```
Error: Generative enum constructors must be marked as 'const'.
```

Es lógico, ya que las *instancias* de un tipo enumerado son las constantes del tipo.

- Al final del constructor de la última constante del tipo se escribe punto y coma `;`. Si no se escribe nada o se escribe una coma `,` se obtiene el error de compilación:

Error: Expected '}' before this.

Importante: En realidad, es obligatorio poner punto y coma `;` siempre al final de la declaración de la última constante, cuando en el tipo enumerado se definan métodos.

Añadir métodos al tipo enumerado

Además del constructor generativo visto en el apartado anterior, también se pueden añadir otros métodos a un `enum`. Por ejemplo si se tiene el tipo enumerado:

```
enum DiaSemana {  
    lunes,  
    martes,  
    miercoles,  
    jueves,  
    viernes,  
    sabado,  
    domingo,  
}
```

se puede sobrecargar el operador de suma `+` para hacer cálculos sencillos con los días de la semana. Si se sustituye la declaración anterior del tipo enumerado por el siguiente fragmento de código:

```
enum DiaSemana {  
    lunes,  
    martes,  
    miercoles,  
    jueves,  
    viernes,  
    sabado,  
    domingo;  
  
    DiaSemana operator +(int dias) {  
        final numeroDeConstantes = DiaSemana.values.length;  
        final indice = (this.index + dias) % numeroDeConstantes;  
        return DiaSemana.values[indice];  
    }  
}
```

dado un día, averiguar los días próximos es sencillo. El siguiente fragmento de código:

```
var diaSemana = DiaSemana.martes;  
diaSemana = diaSemana + 2;  
print(diaSemana);  
diaSemana += 4;  
print(diaSemana);  
diaSemana++;  
print(diaSemana);
```

imprime en la consola:

```
DiaSemana.jueves  
DiaSemana.lunes  
DiaSemana.martes
```

En Dart, cuando se sobrecarga el operador suma `+` también se tienen los operadores `+=` y `++` **sin tener que codificar nada**.

Por supuesto también se podría haber sobrecargado el operador `-` o cualquier otro que tuviera sentido en un tipo enumerado concreto.

Veamos con un poco de detalle la implementación del operador `+` sobrecargado:

- Los días de la semana son cíclicos, es decir, tras el domingo va de nuevo el lunes, o al contrario, delante del lunes viene el domingo. Esto puede programarse con una suma módulo el número de elementos a recorrer.
Esa es la razón de que lo primero que se hace es calcular el número de constantes en el tipo enumerado. Para ello se usa la propiedad de clase `values` que devuelve una lista con todas las constantes del tipo, y a continuación se usa la propiedad `length` de las listas que devuelve su número de elementos.
El valor `numeroDeConstantes` es el que se usa para calcular el módulo.
- Lo siguiente que se hace es averiguar el índice que indica la posición de la constante que corresponde con el valor que se calcula. Para ello, al índice de la constante sobre la que se aplica la suma, se le suma el número de días a desplazar y a continuación se calcula el módulo según el número de constantes del tipo, para que si el valor se sale del rango, vuelva al inicio.
- Por último se averigua a qué día de la semana corresponde el índice calculado y se devuelve.

Una vez que se ha sobrecargado el operador de suma, también se puede añadir otro método de utilidad como por ejemplo:

```
DiaSemana next() {  
    return this + 1;  
}
```

que se puede usar de la siguiente forma:

```
var diaSemana = DiaSemana.martes;  
print(diaSemana.next());
```

con el siguiente resultado en la consola:

```
DiaSemana.miercoles
```

Nota: También se podría haber implementado `next` como una propiedad calculada en lugar de como un método.

Importante: A partir de la versión 2.17 de Dart, los enums se pueden tratar como clases. Eso significa que se les puede añadir propiedades y métodos como a cualquier otra clase. Eso hace que sean muy potentes. Cuando programemos aplicaciones móviles en Flutter, usaremos mucho los tipos enumerados mejorados.

Mini ejercicios

1. ➡ Añade a la jerarquía de clases definida por la clase `Spaceship`, otra clase de naves espaciales de tu elección. Añádele las propiedades y métodos necesarios para especializar a `Spaceship`

Pruébala declarando algunas naves de esa subclase.

2. Las funciones trigonométricas de Dart, que se encuentran en la biblioteca `dart:math` esperan sus argumentos expresados en radianes, sin embargo para un usuario humano, es más cómodo expresar los ángulos en grados sexagesimales.

Escribe una extensión para añadir a Dart la capacidad de pasar de grados sexagesimales a radianes y viceversa.

- ¿A qué clase le has añadido esa extensión? ¿por qué?
- ¿Has definido propiedades calculadas o métodos? ¿por qué?

3. Al operar con valores de tipo `double` suelen producirse errores de redondeo que pueden dar lugar a errores. Uno de los más comunes se da al comparar si un valor de tipo `double` es cero. En este caso podría darse el caso de que en una variable, por errores de redondeo en las operaciones previas, no se tenga el valor `0.0` sino uno muy próximo como por ejemplo: `0.000000001`. En ese caso la comparación `== 0.0` devuelve `false`

Para evitar esos errores, al comparar si un número real es cero, lo que se hace es definir una constante `epsilon` arbitrariamente pequeña, por ejemplo: `0.000001` y al comparar una variable de tipo `double` por ejemplo: `valorDouble` no usar:

```
if (valorDouble == 0.0) {  
    // Aquí el resto del código  
}
```

sino:

```
if (abs(valorDouble) <= epsilon) {  
    // Aquí el resto del código  
}
```

Escribe una extensión para la clase `double` que permita hacer comparaciones seguras con cero de forma sencilla. Explica con detalle cómo lo has hecho y por qué.

4. 🕒 Inspirándote en el ejercicio anterior, escribe una extensión para la clase `double` que, además de comparaciones seguras con cero, permita comparaciones seguras entre cualesquiera dos valores reales.

Pista: dos valores reales son iguales si su distancia es menor o igual que `epsilon`

5. 🕒 Dart dispone de muchas colecciones muy útiles, pero carece de una colección de tipo **LIFO** (Last In, First Out)

Escribe una clase genérica `Stack` que encapsule el comportamiento de una colección de tipo LIFO que pueda contener elementos de cualquier tipo `T` que se necesite. Añade a la clase las propiedades:

- `bool isEmpty` Que devuelva `true` si la pila está vacía y `false` si tiene elementos.
- `T? peek` que devuelva el elemento que ocupa el tope de la pila, pero sin sacarlo de la pila. ¿Por qué crees que el tipo devuelto se ha marcado como `T?` y no como `T` solamente?

y los métodos:

- `void push(T elemento)` Que añada el `elemento` que se pasa como parámetro a la pila.
- `T? pop()` Que quite el elemento que ocupa el tope de la pila y lo devuelva.
- `void clear()` Que vacíe completamente la pila.

Sobreescribe también el método `toString` para poder imprimir la pila de forma sencilla.

6. Sobrecarga el operador `-` para el tipo enumerado `DiaSemana` declarado en este guión de prácticas. Pruébalo con algunos cálculos de días de semana.
7. Una vez sobrecargado el operador `-` en el ejercicio anterior, úsalo para añadir una propiedad calculada `previous` al tipo enumerado `DiaSemana` de forma que cuando se aplique a un día de la semana, devuelva el día anterior. Pruébalo con algunos cálculos de días de semana.
8. 🕒 Las constantes de un tipo enumerado pueden escribirse en la consola directamente con la orden `print`, pero el mensaje que se obtiene no es muy apropiado para usar en una aplicación móvil. Por ejemplo:

```
var diaSemana = DiaSemana.martes;  
print(diaSemana);
```

muestra en la consola:

```
DiaSemana.martes
```

Modifica el tipo enumerado `DiaSemana` para que al imprimirlo muestre un nombre con sentido para un usuario. Por ejemplo, en el caso anterior que imprima `Martes`